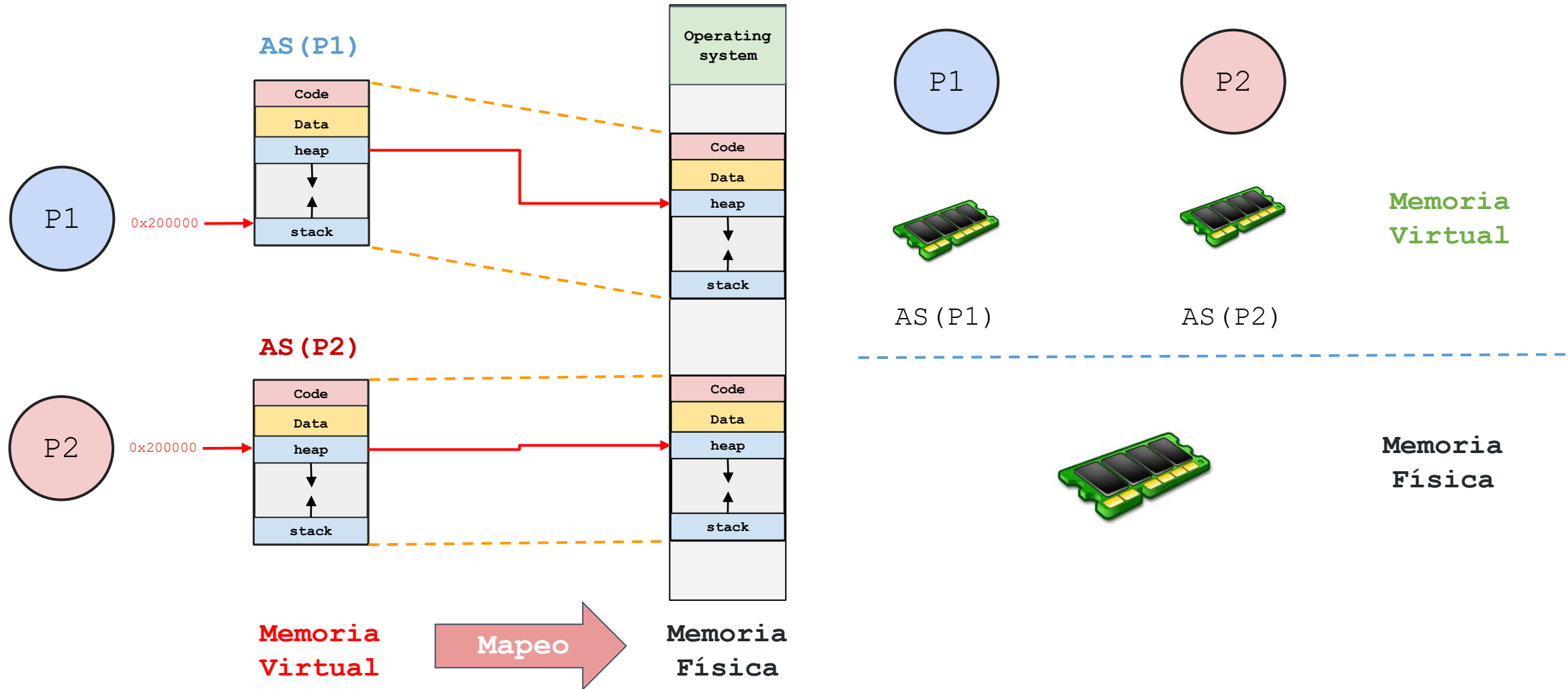


# Sistemas Operativos

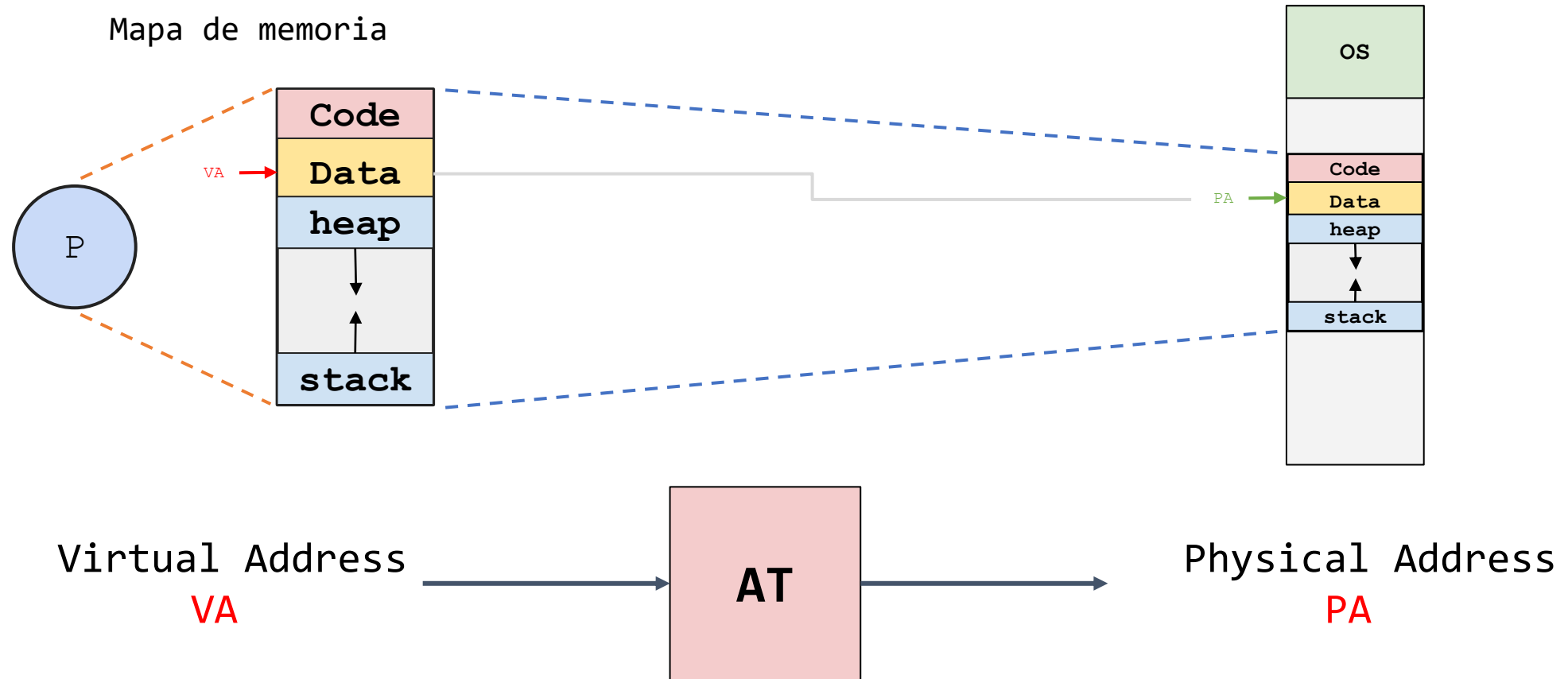
## Clase 11 – TLB

14/04/2026

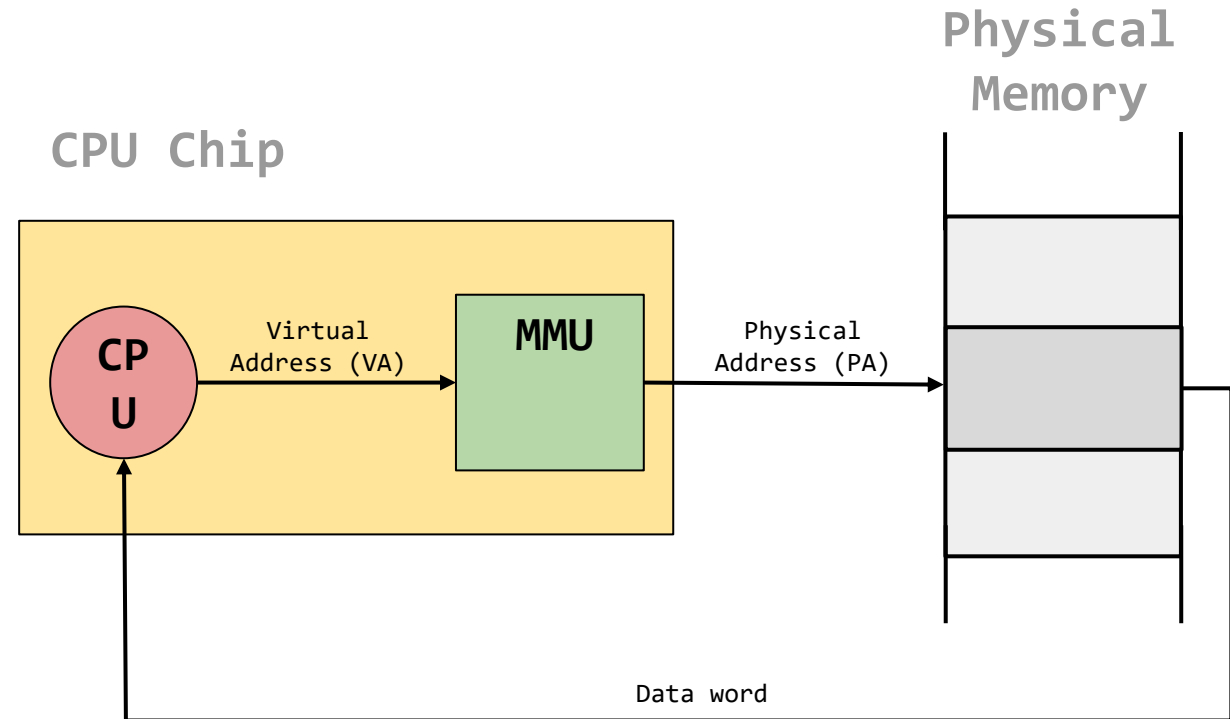
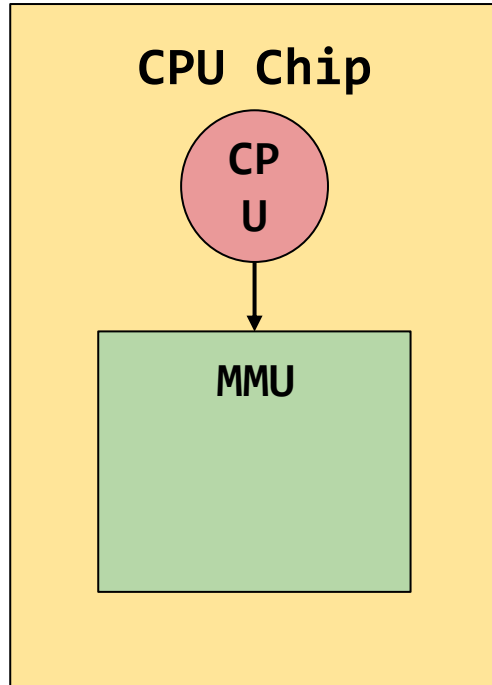
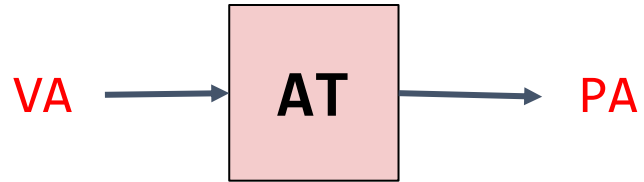
# Virtualización de memoria



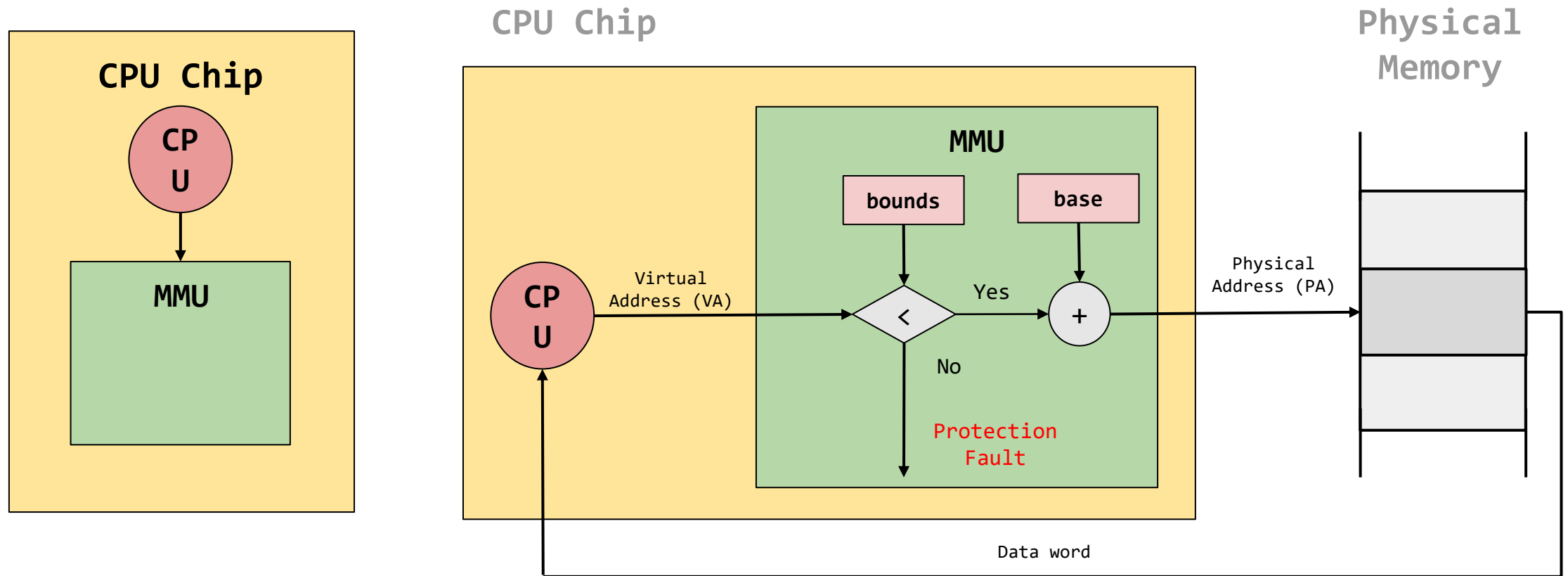
# Address Translation - Mapeo



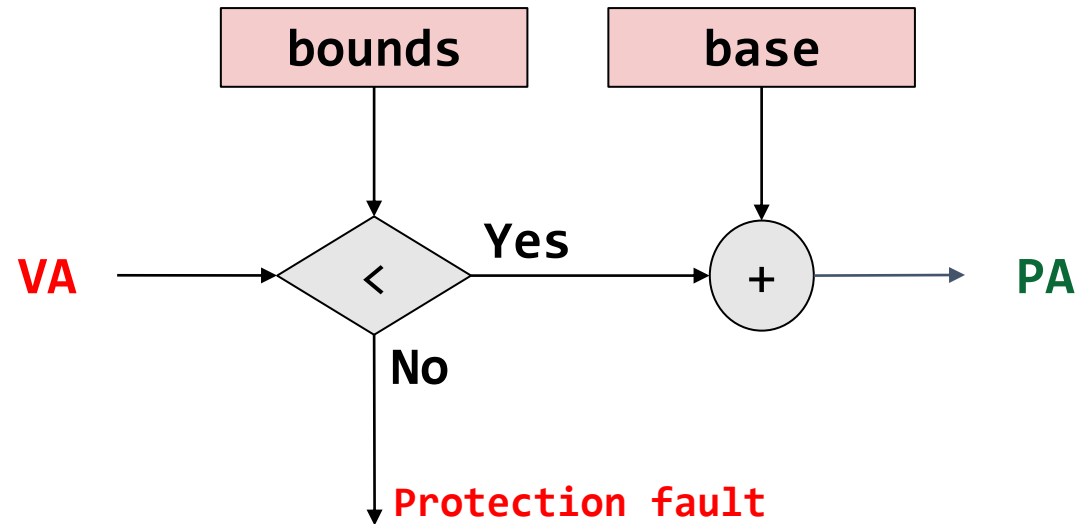
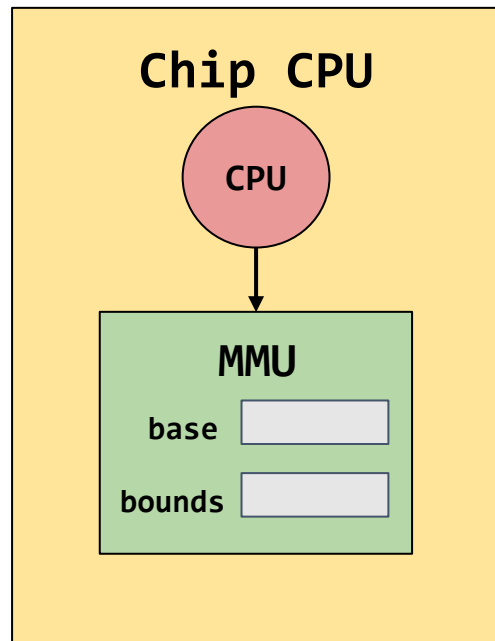
# Memory Management Unit (MMU)



# Traducción de direcciones – Dynamic Relocation (Base & Bounds)



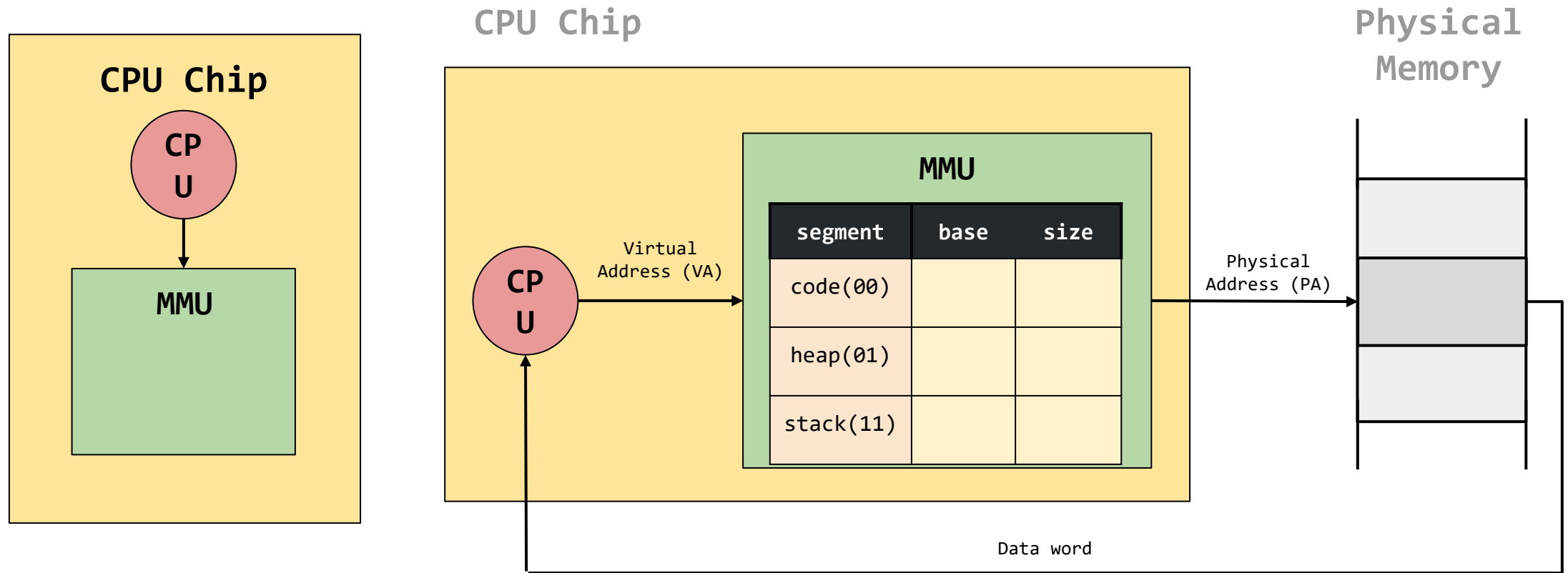
# Traducción de direcciones – Dynamic Relocation (Base & Bounds)



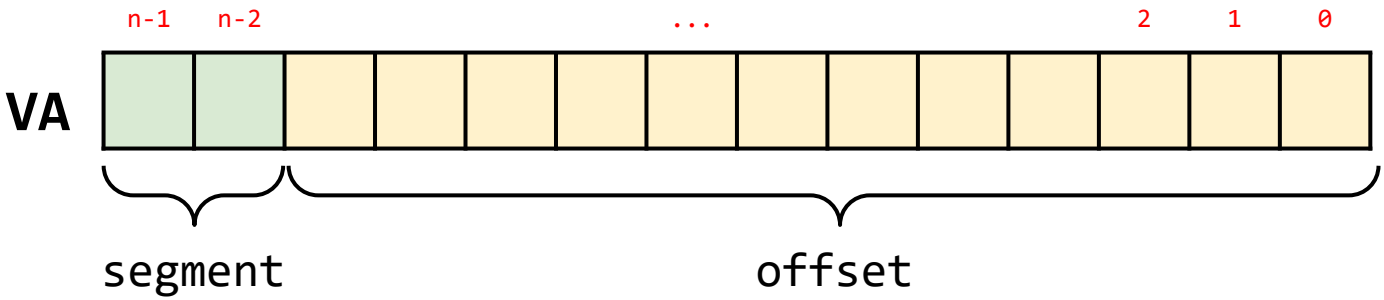
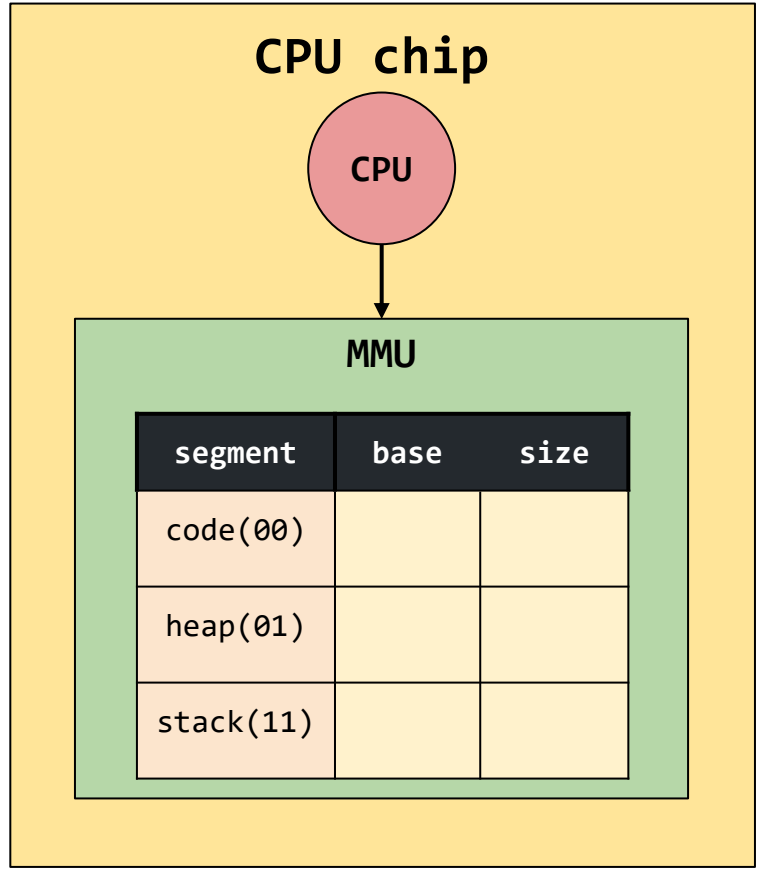
$$PA = VA + base$$

$$0 \leq VA < bounds$$

# Traducción de direcciones – Segmentation

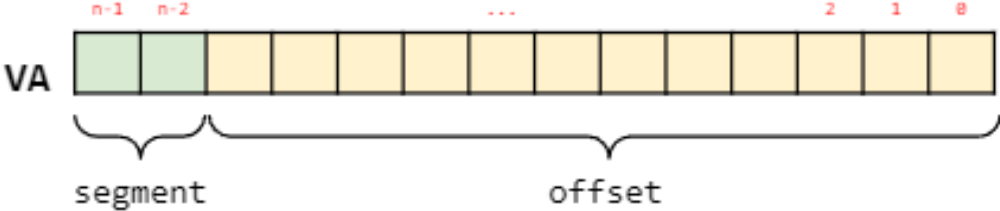


# Traducción de direcciones – Segmentation

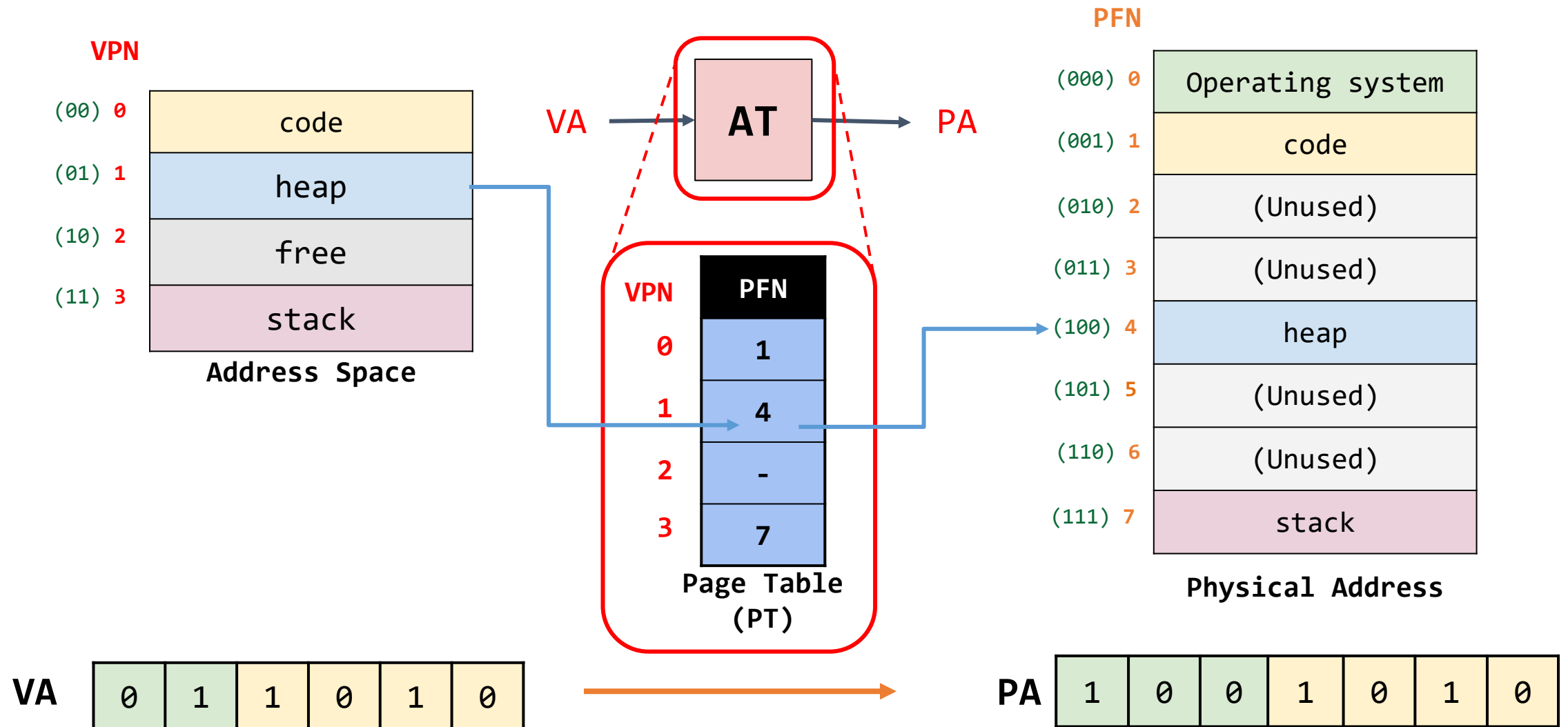


| segment   | base | size | Grows Positive? | Protection   |
|-----------|------|------|-----------------|--------------|
| code(00)  |      |      |                 | Read-Execute |
| heap(01)  |      |      |                 | Read-Write   |
| stack(11) |      |      |                 | Read-Write   |

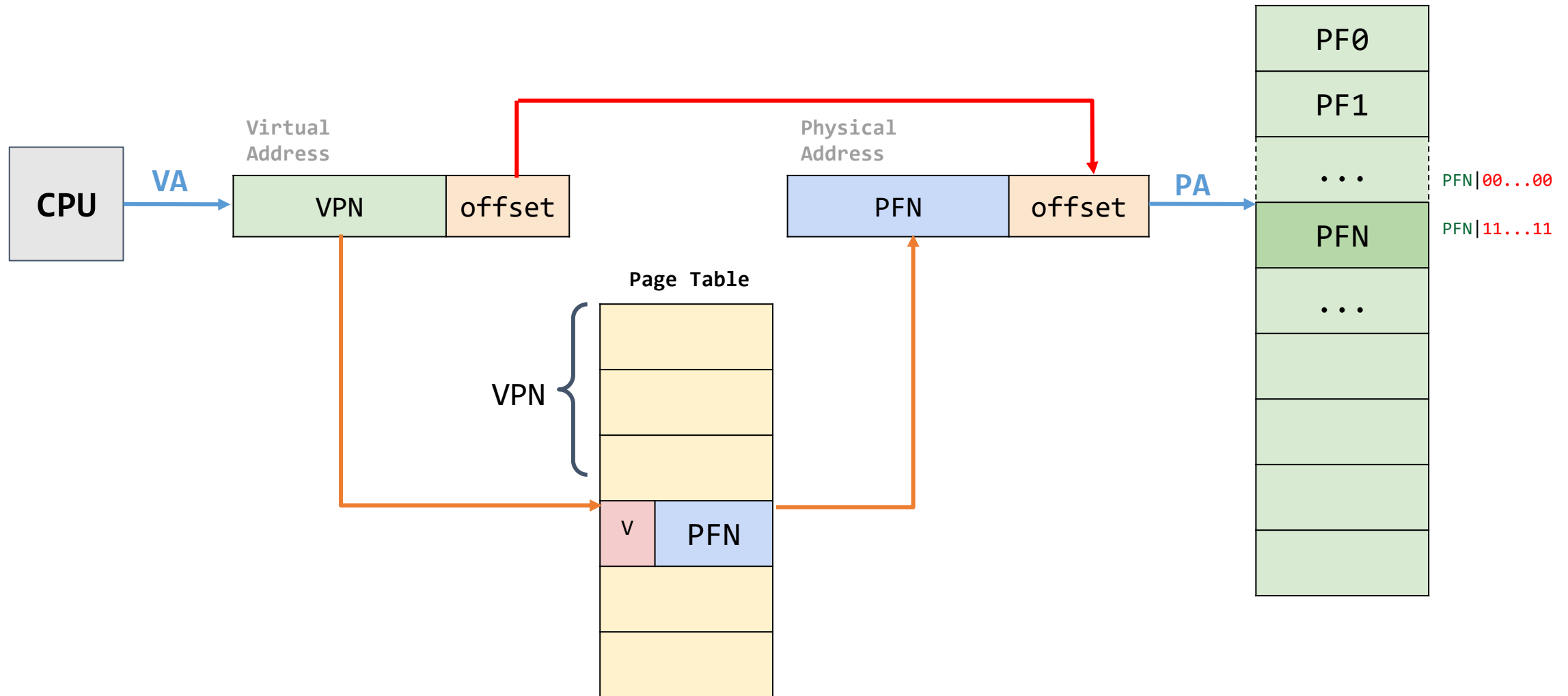
# Traducción de direcciones – Segmentation

| segment | offset                                                                                                                                                                                   | Physical Address                                        |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| code    | $\text{offset} = \text{VA}$                                                                                                                                                              | $\text{PA} = \text{offset} + \text{base}(\text{code})$  |
| heap    | $\text{offset} = \text{VA} - \text{VA}(\text{heap})$                                                                                                                                     | $\text{PA} = \text{offset} + \text{base}(\text{heap})$  |
| stack   | <div><p><math>\text{offset}(\text{stack}) = \text{offset} - \text{offset}(\text{max})</math></p></div> | $\text{PA} = \text{offset} + \text{base}(\text{stack})$ |

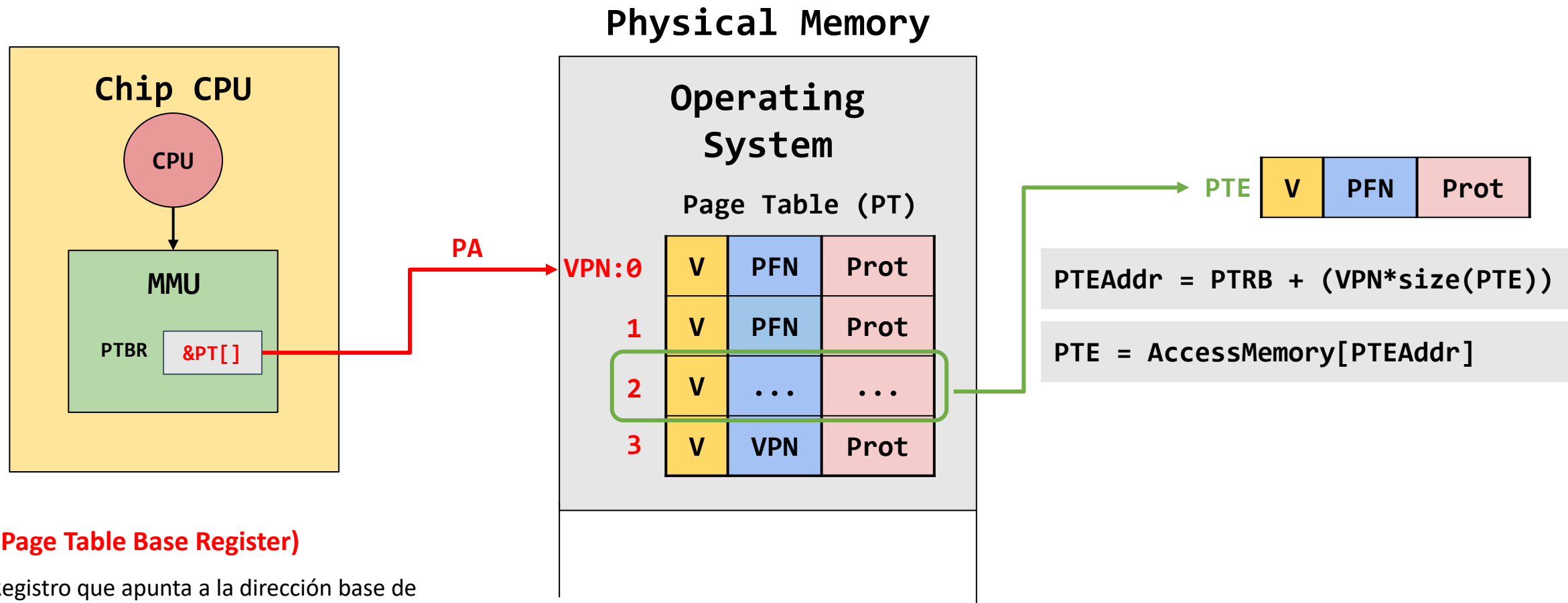
# Traducción de direcciones – Paginación



# Traducción de direcciones – Paginación



# Traducción de direcciones – Paginación



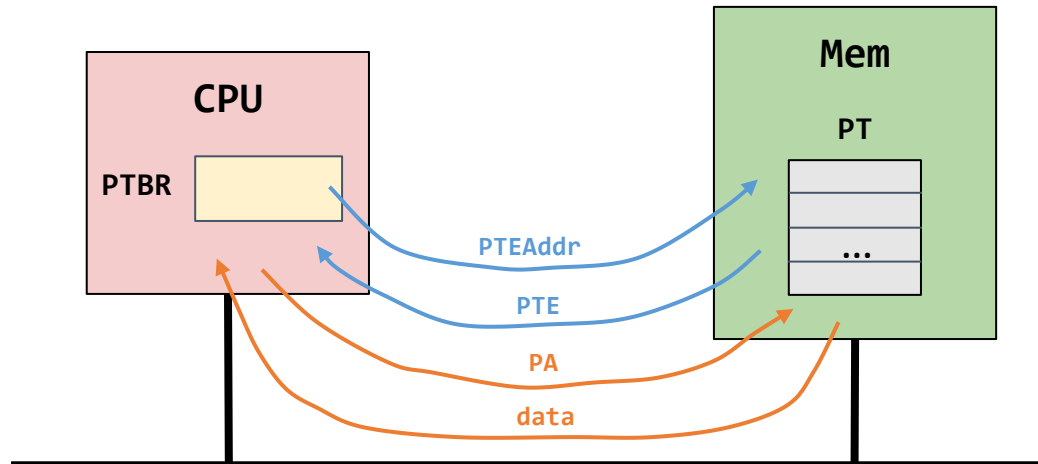
## PTBR (Page Table Base Register)

- Registro que apunta a la dirección base de la tabla de página del proceso que se encuentra actualmente en ejecución.
- Cada proceso tiene su propio PTBR.

# Traducción de direcciones – Paginación

## Pasos de la traducción de direcciones mediante paginación

1. Extraer el VPN de la VA [cheap]
2. Calcular la dirección del PTE (PTEAddr) [cheap]
3. Obtener (Fetch) el PTE [cost]
4. Extraer el PFN [cheap]
5. Construir la PA [cheap]
6. Traer el PA a un registro [cost]



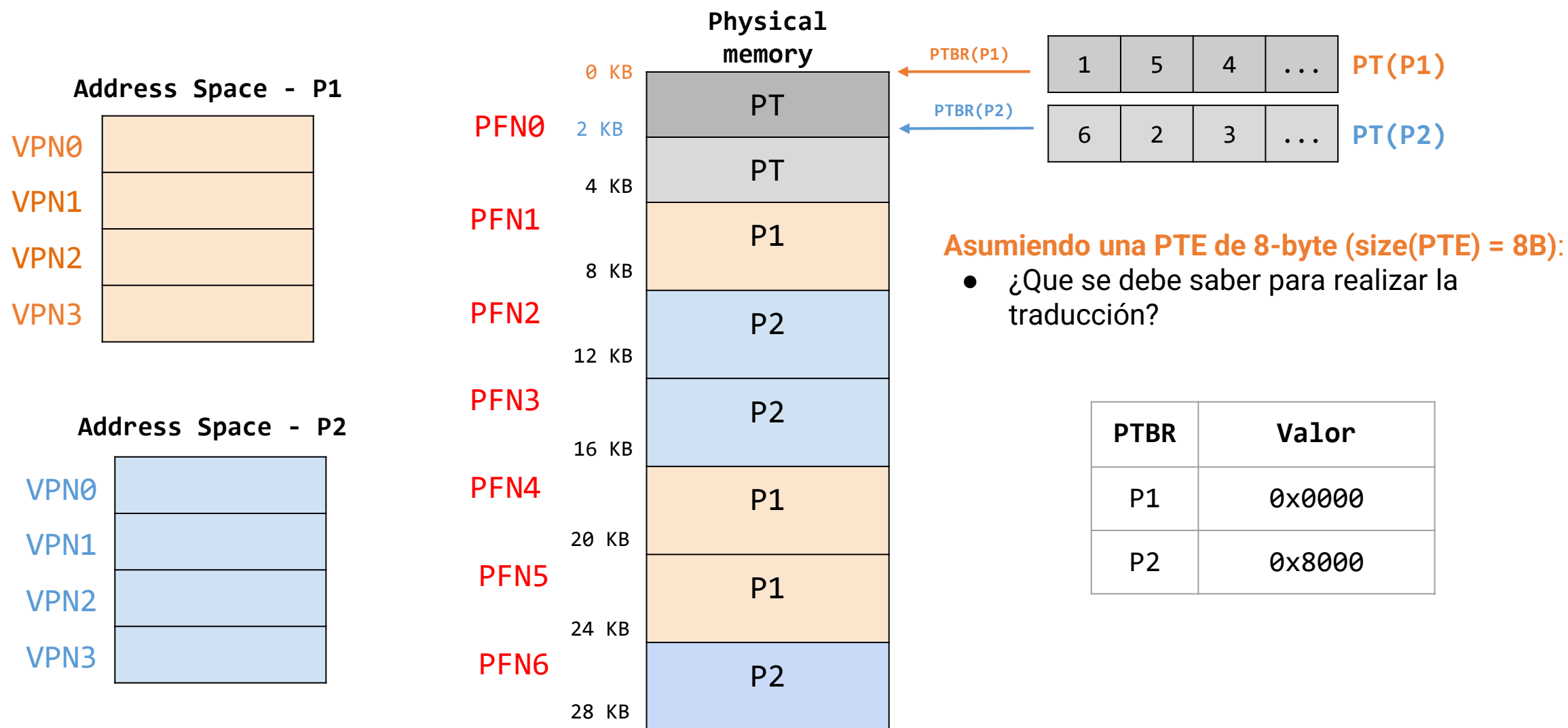
## Acceso a memoria

Cada instrucción genera dos referencias a memoria

1. Una a la PT (Page table) para encontrar el PFN donde se encuentra la instrucción.
2. Para traer de memoria (fetch) la instrucción como tal.

# Traducción de direcciones – Paginación

## Traducción de direcciones - Ejemplo paginación



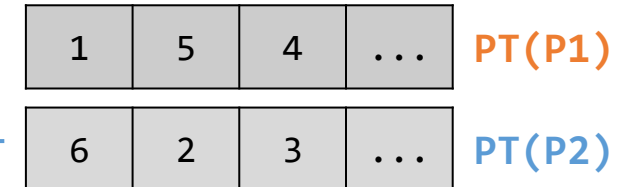
# Traducción de direcciones – Paginación

## Traducción de direcciones - Ejemplo paginación

$$\text{PTEAddr} = \text{PTRB} + (\text{VPN} * \text{size}(\text{PTE}))$$

$$\text{PTBR}(\text{P1}) = 0x0000$$

$$\text{PTBR}(\text{P2}) = 0x0800$$

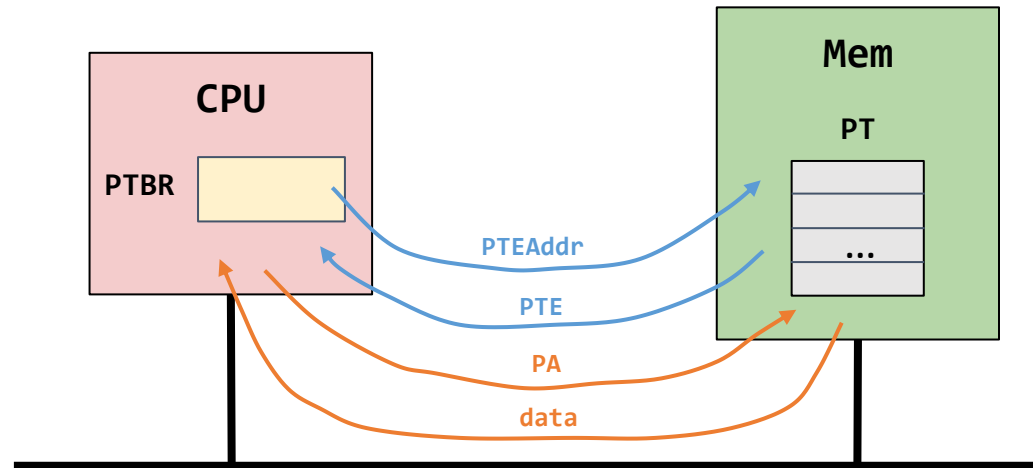


| Proceso | Virtual     | Physical                                      |
|---------|-------------|-----------------------------------------------|
| P2      | load 0x0000 | load 0x0800 : PTEAddr = 0x0800 + 0*8 = 0x0800 |
|         |             | load 0x6000 (PA = 0x6000 = 24 KB)             |
| P2      | load 0x1444 | load 0x0800 : PTEAddr = 0x0800 + 1*8 = 0x0808 |
|         |             | load 0x2444                                   |
| P1      | load 0x1444 | load 0x0000 : PTEAddr = 0x0000 + 1*8 = 0x0008 |
|         |             | load 0x5444                                   |

# Traducción de direcciones – Paginación

## Traducción de direcciones - Problemas de la paginación

1. Lentitud en el proceso debido a los accesos a la tabla de página.
2. Gasto de memoria (cada proceso tiene su propia tabla de página).



# Traducción de direcciones – Trazas de memoria

## Traducción de direcciones - Trazas de memoria

### Código

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

### Compilación y ejecución

```
prompt> gcc -o array array.c -Wall -o  
prompt> ./array
```

### Código ensamblador resultante

```
0x1024 movl $0x0, (%edi,%eax,4)  
0x1028 incl %eax  
0x102c cmpl $0x03e8,%eax  
0x1030 jne 0x1024
```

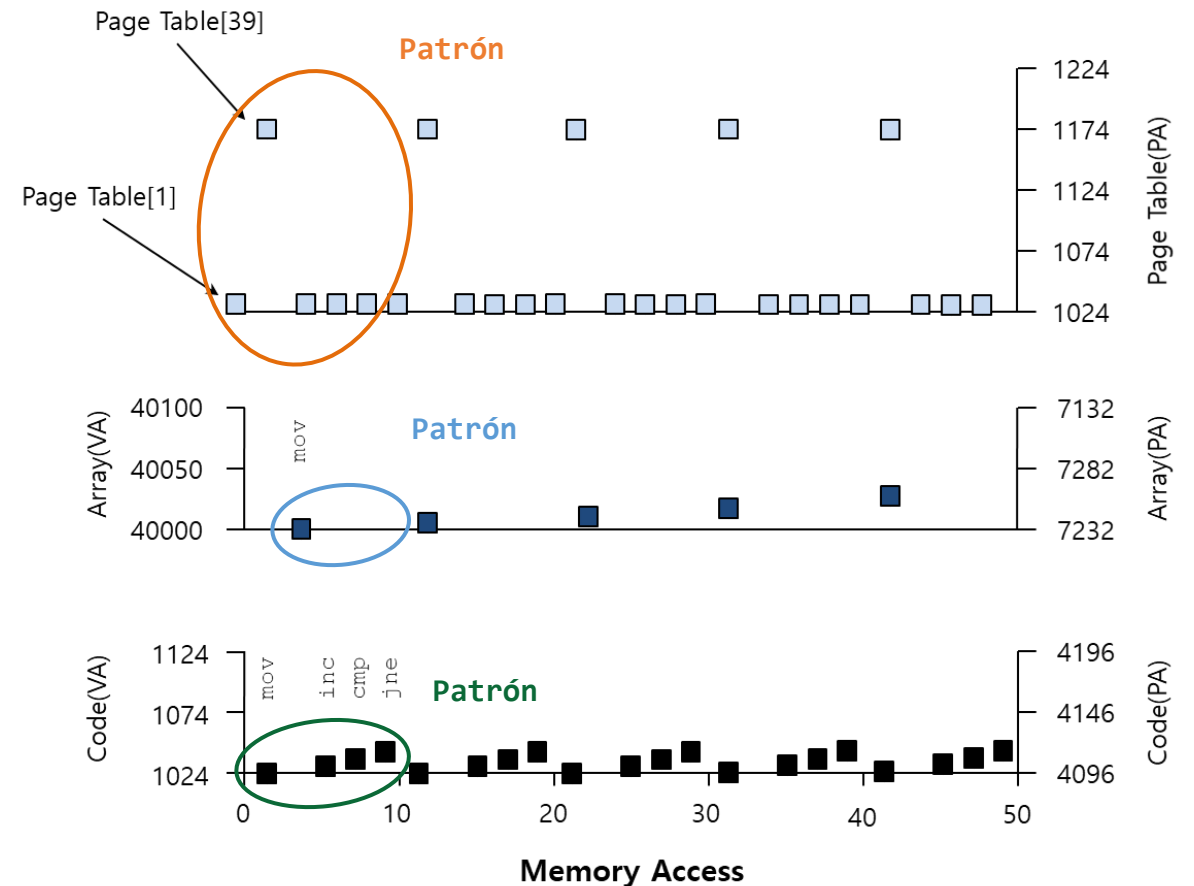
# Traducción de direcciones – Trazas de memoria

```
0x1024 movl $0x0, (%edi,%eax,4)
0x1028 incl %eax
0x102c cmpl $0x03e8,%eax
0x1030 jne 0x1024
```

Análisis:  
5 ciclos

5 ciclos \* 10 accesos/1 ciclo = **50 accesos**

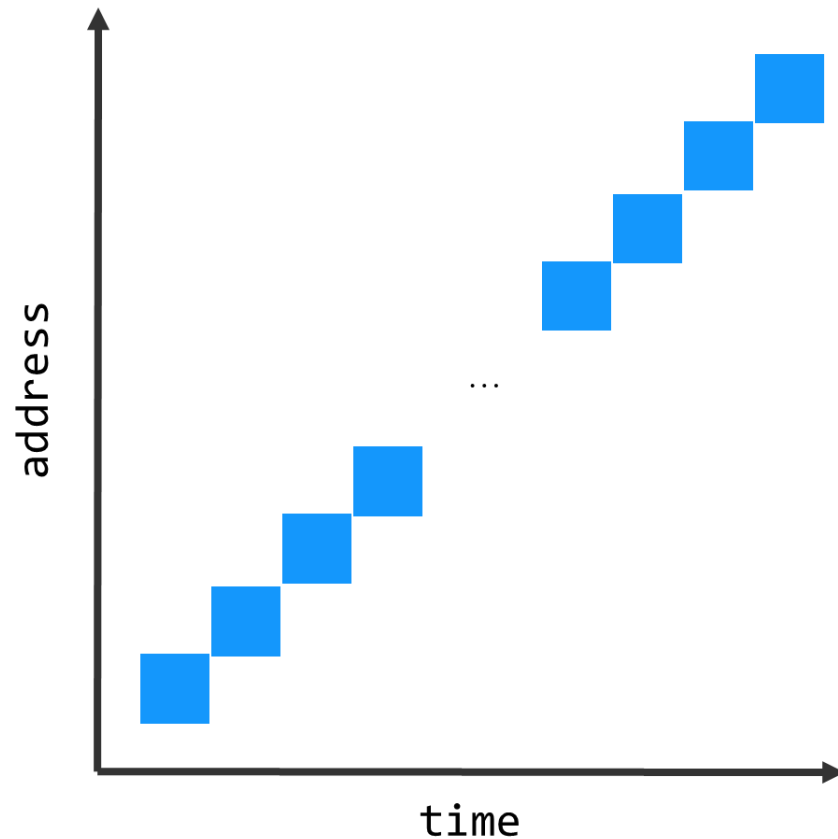
| Instrucción               | Fetch inst | Mem Access |
|---------------------------|------------|------------|
| movl \$0x0, (%edi,%eax,4) | 2          | 2          |
| incl %eax                 | 2          |            |
| cmpl \$0x03e8,%eax        | 2          |            |
| jne 0x1024                | 2          |            |



# Traducción de direcciones – Trazas de memoria

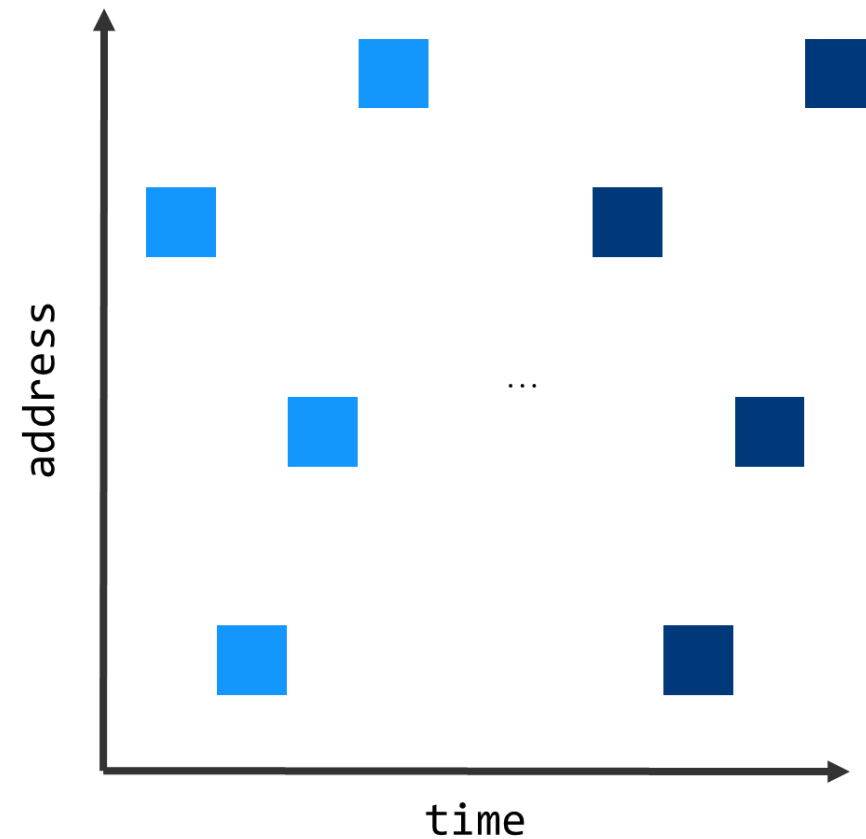
## Localidad espacial

Acceso secuencial



## Localidad Temporal

Acceso aleatorio repetido

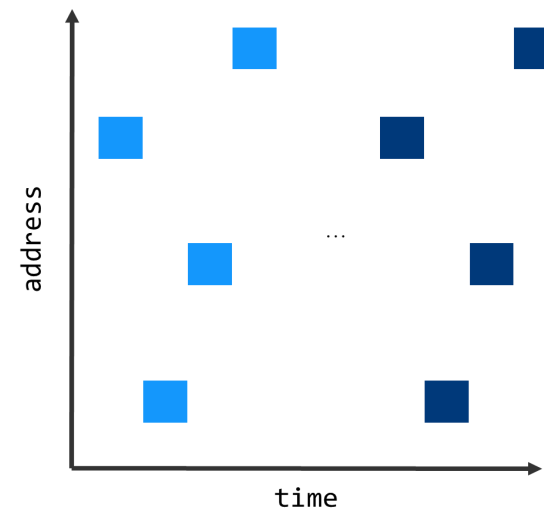
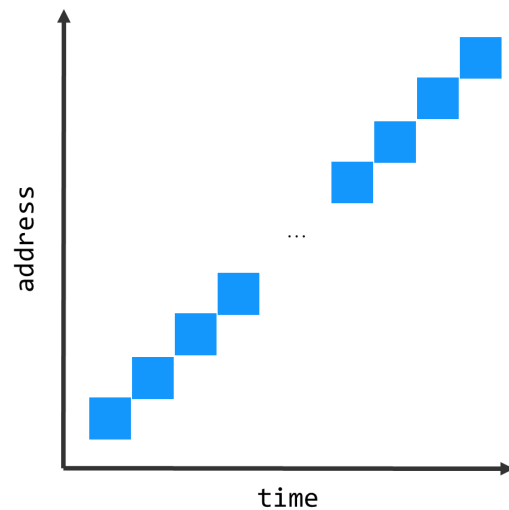


# Contextualización

## Pregunta clave

### ¿Cómo mejorar la traducción de direcciones?

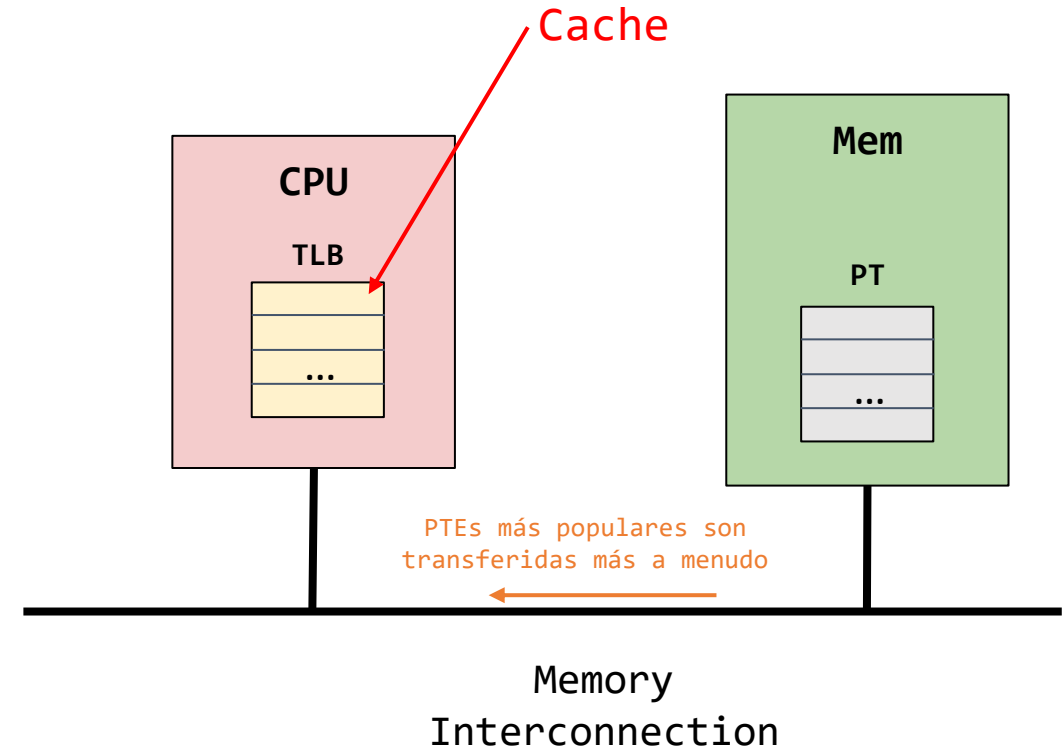
- ¿Cómo podemos acelerar la traducción de direcciones evitando la referencia adicional a memoria necesaria cuando se usa paginación?
- ¿Que soporte de hardware es necesario?
- ¿Qué es lo que tiene que hacer el sistema operativo en este caso?



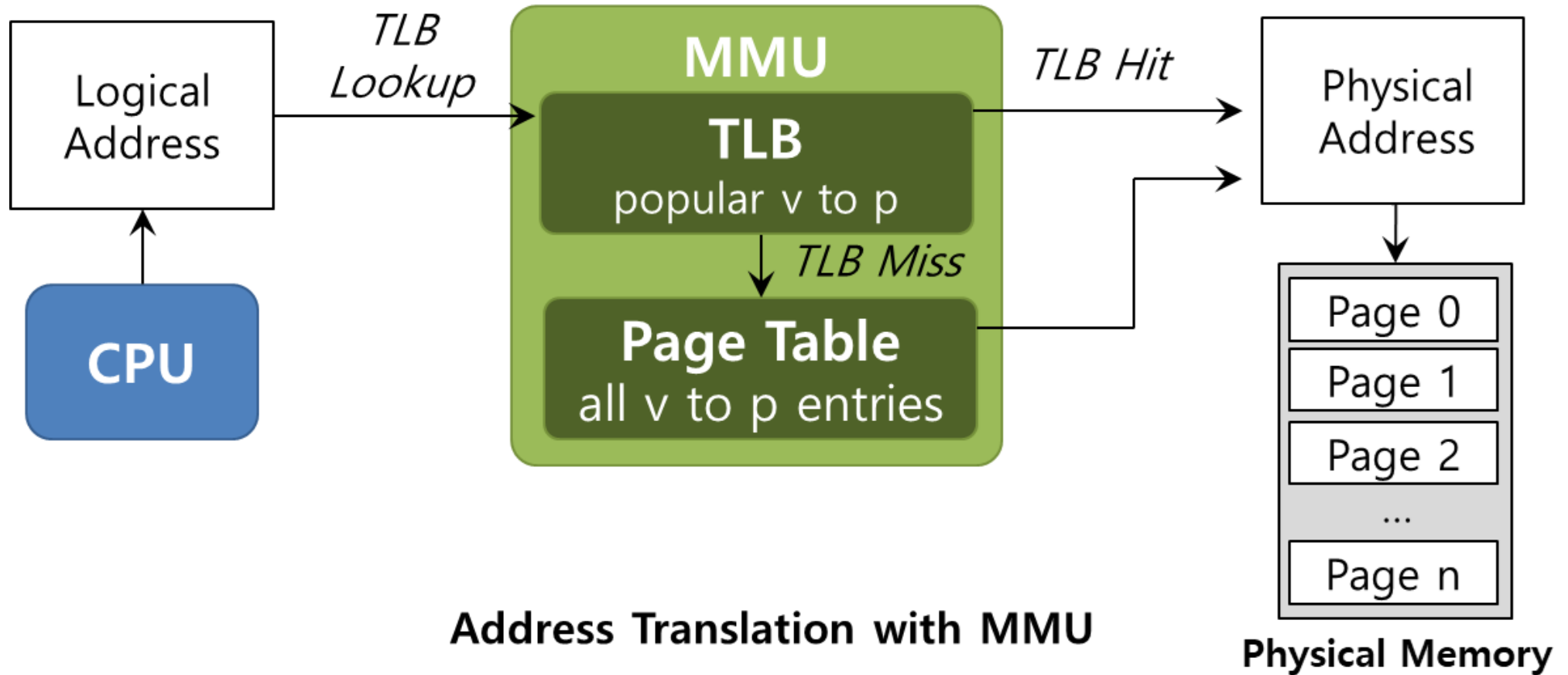
# TLB (Translation Lookaside Buffer)

## Aspectos claves

- La **TLB** es una memoria cache on chip (En la CPU).
  - Pequeña y rápida.
  - Mantiene las direcciones más populares de la tabla de página (PT).
  - Tamaños típicos: 16 ~ 256 entradas.
  - Usualmente es full asociativo (todas las entradas son comparadas en paralelo), pero puede haber asociatividad para reducir la latencia.



# TLB (Translation Lookaside Buffer)

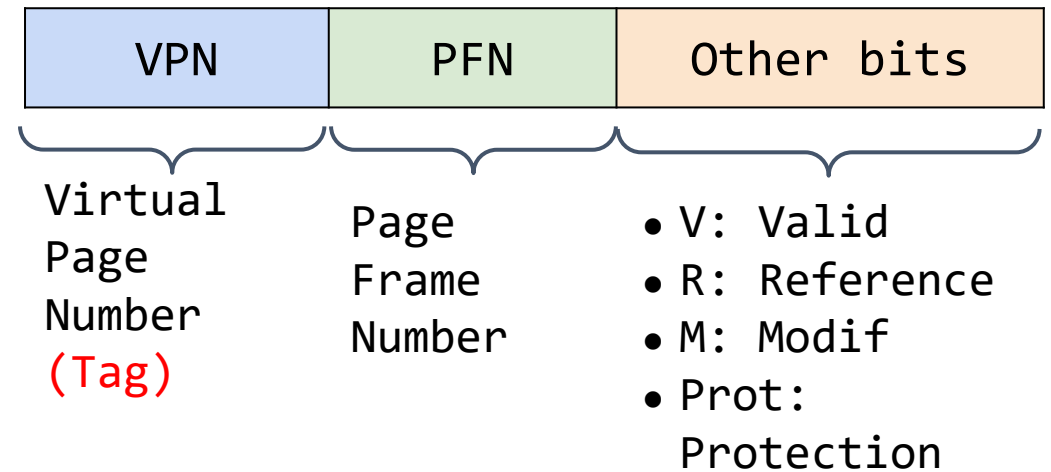


# TLB (Translation Lookaside Buffer)

## Contenido de una entrada TLB

- A diferencia de una entrada de una tabla de página (**PTE = PFN + Other bits**), una entrada de una TLB (**TLB Entry**) contiene la siguiente forma:

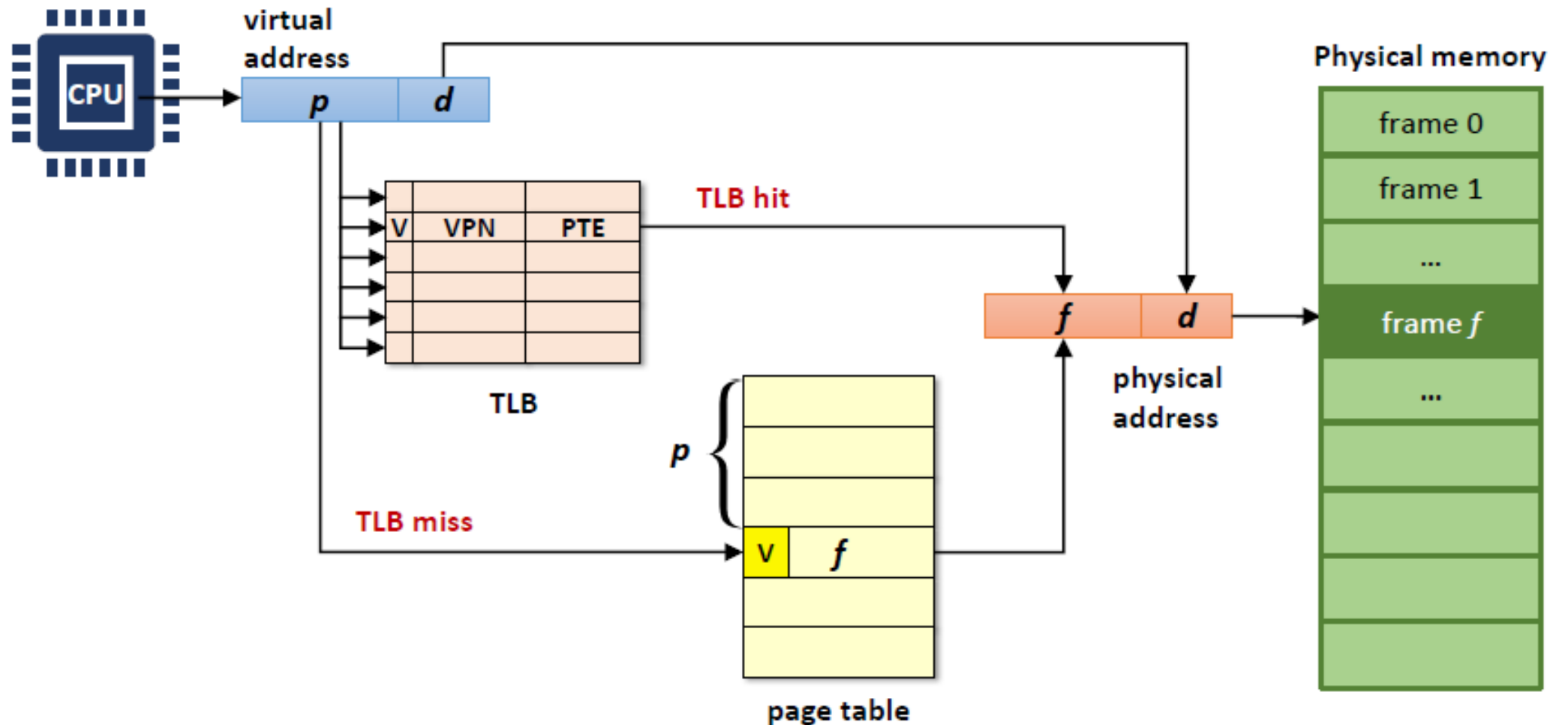
**TLB Entry = VPN + PFN + Other bits**



| Valid | Tag (VPN) | Value (PTE) |   |   |      |     |        |
|-------|-----------|-------------|---|---|------|-----|--------|
| 1     | 0x1000    | V           | R | M | Prot | PFN | 0x1234 |
| 1     | 0x2400    | V           | R | M | Prot | PFN | 0x8800 |
| 0     | -         | -           |   |   |      |     |        |

# Address Translation usando TLB

## Hardware de traducción con TLB

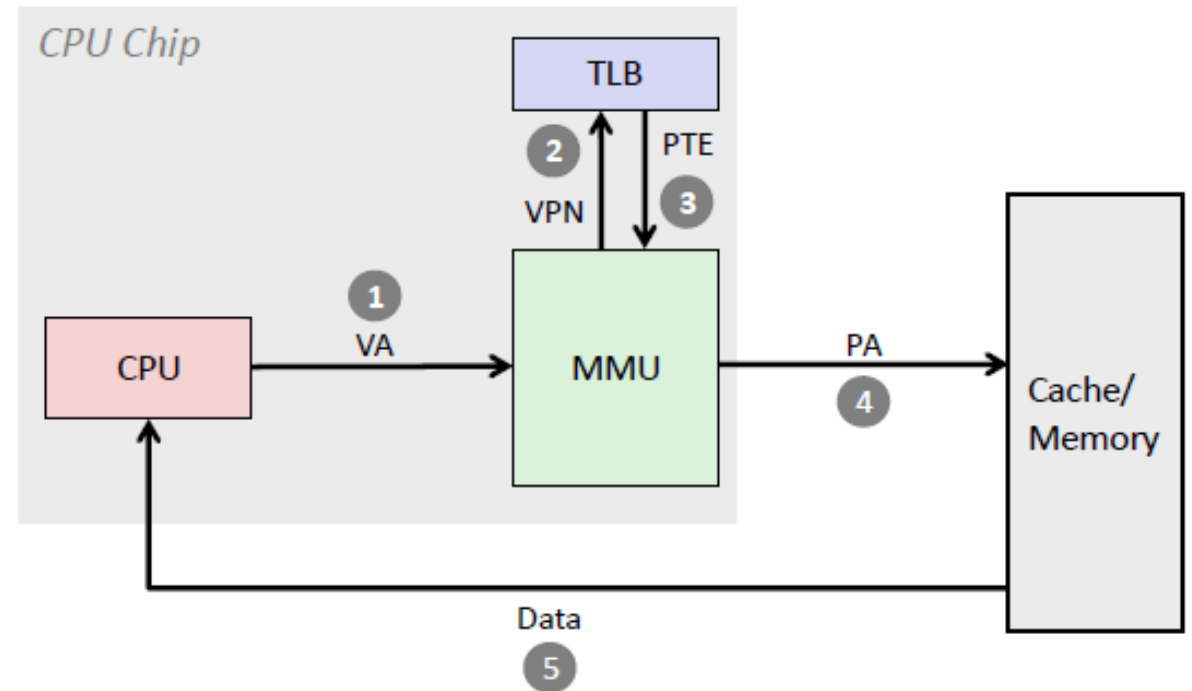


# Address Translation usando TLB

## Caso Hit

Se elimina la necesidad de acceder a memoria física:

1. El procesador envía la VA a la MMU.
2. La MMU extrae la VPN de la VA.
3. Se verifica que la VPN se encuentre en la TLB, como en este caso esto es verdadero (**TLB hit**), se obtiene la PTE.
4. Se extrae la PFN de la PTE y se procede a formar la dirección física (PA) asociada a la dirección virtual.
5. Con la dirección física obtenida, se procede a cargar el dato que allí se encuentra a la CPU.

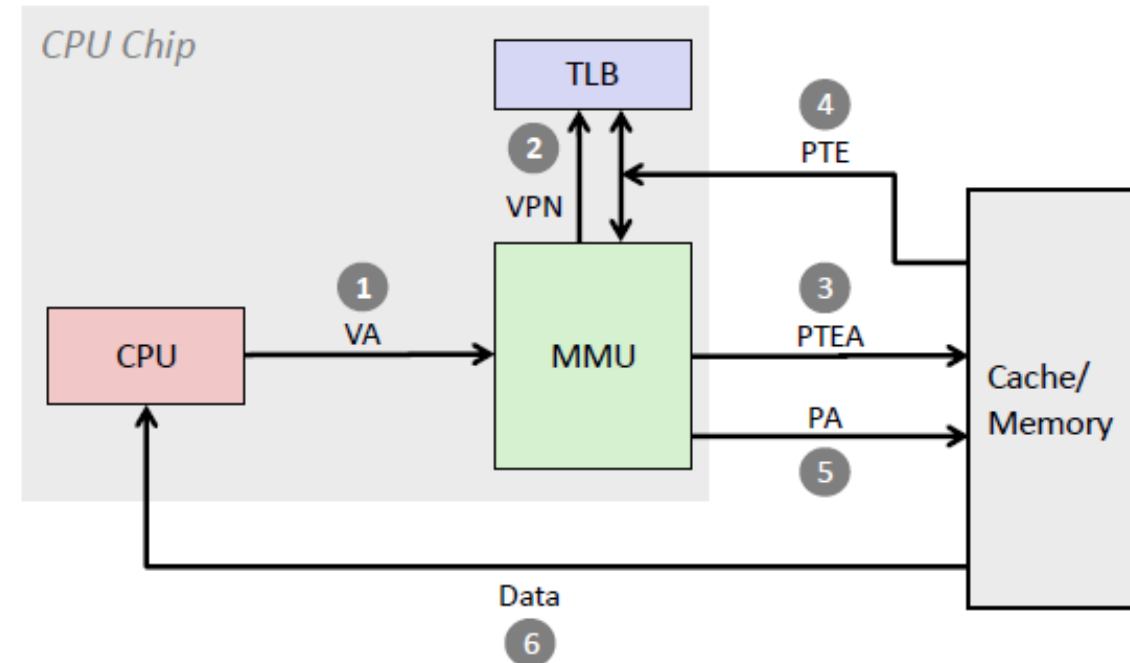


# Address Translation usando TLB

## Caso Miss

En este caso, se da un acceso adicional a memoria (más exactamente a la PT). En programas con buena localidad, los TLB misses son poco frecuentes:

1. El procesador envía la VA a la MMU.
2. La MMU extrae la VPN de la VA.
3. Se verifica que la VPN se encuentre en la TLB, como en este caso no se encuentra (**TLB miss**), se tiene que acceder a la PT.
4. Una vez al obtiene el PFN (al acceder a la PT) asociado al VPN deseado, se procede a la actualización de la TLB.
5. Se realiza nuevamente la búsqueda original en la TLB ya actualizada produciéndose un TLB Hit lo cual hace posible formar la dirección física deseada.
6. Con la dirección física obtenida, se procede a cargar el dato que allí se encuentra a la CPU.



# Address Translation usando TLB

## Control Flow Basic Algorithm (Pseudocódigo)

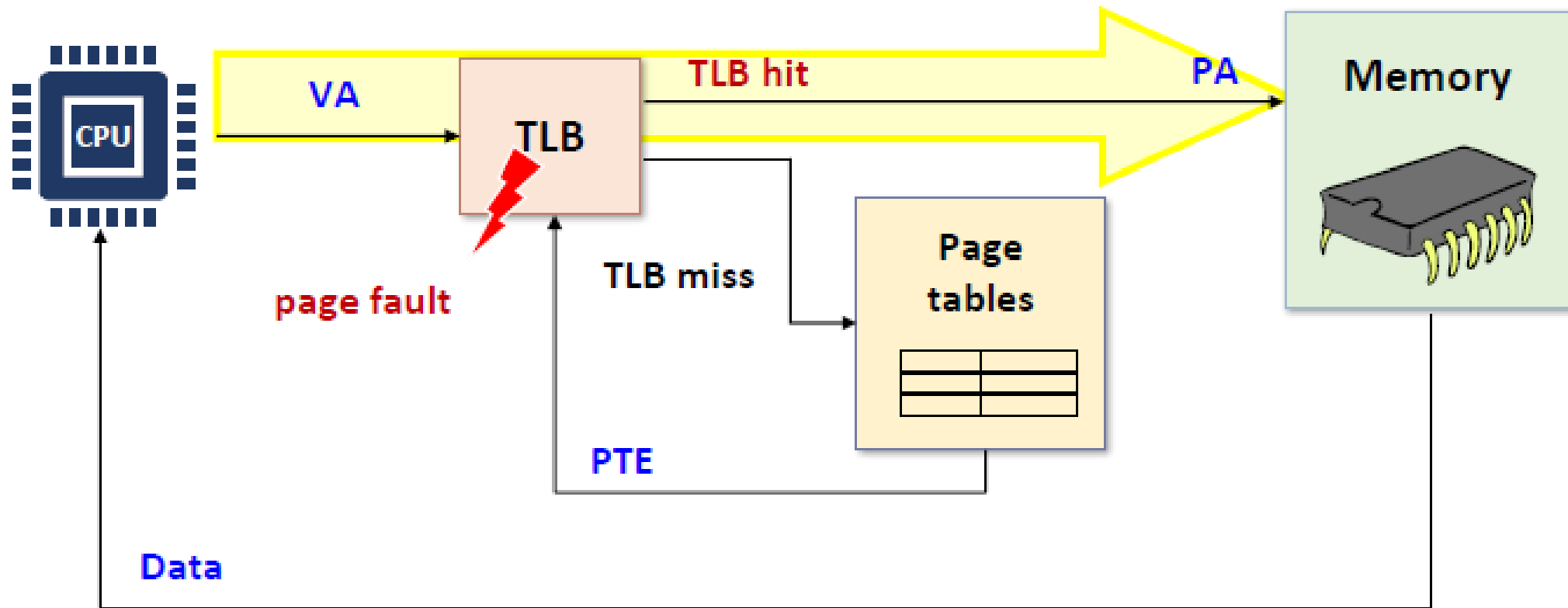
```
1  VPN = (VirtualAddress & VPN MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True) // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```

### Resumen Algoritmo:

- Instrucciones comunes (■)
- Instrucciones hit (■)
- Instrucciones miss (■)
- Instrucciones interrupción (■)

# Address Translation using TLB

## Control Flow



# Address Translation usando TLB

|          | OFFSET |      |      |      |    |
|----------|--------|------|------|------|----|
|          | 00     | 04   | 08   | 12   | 16 |
| VNP = 00 |        |      |      |      |    |
| VNP = 01 |        |      |      |      |    |
| VNP = 02 |        |      |      |      |    |
| VNP = 03 |        |      |      |      |    |
| VNP = 04 |        |      |      |      |    |
| VNP = 05 |        |      |      |      |    |
| VNP = 06 |        | a[0] | a[1] | a[2] |    |
| VNP = 07 | a[3]   | a[4] | a[5] | a[6] |    |
| VNP = 08 | a[7]   | a[8] | a[9] |      |    |
| VNP = 09 |        |      |      |      |    |
| VNP = 10 |        |      |      |      |    |
| VNP = 11 |        |      |      |      |    |
| VNP = 12 |        |      |      |      |    |
| VNP = 13 |        |      |      |      |    |
| VNP = 14 |        |      |      |      |    |
| VNP = 15 |        |      |      |      |    |

## Ejemplo - Accediendo a un arreglo

A continuación se muestra como una TLB puede mejorar el desempeño en la traducción de direcciones

```
1  int sum = 0;
2  for (i = 0; i < 10; i++) {
3      sum += a[i];
4  }
```

## Suposiciones (continuación):

1. Se tiene un array de diez elementos tipo int:
  - $\text{size}(\text{int}) = 4\text{B} \rightarrow \text{size}(\text{array}) = 40\text{B}$
2. La dirección base del arreglo a es 100:  $\&a[0] = 100$
3. Address Space:
  - Número de bits de direcciones:  $n(\text{VA}) = 8$
  - Tamaño de la memoria:  $2^8 = 256$

# Address Translation usando TLB

|          | OFFSET |      |      |      |    |
|----------|--------|------|------|------|----|
|          | 00     | 04   | 08   | 12   | 16 |
| VPN = 00 |        |      |      |      |    |
| VPN = 01 |        |      |      |      |    |
| VPN = 02 |        |      |      |      |    |
| VPN = 03 |        |      |      |      |    |
| VPN = 04 |        |      |      |      |    |
| VPN = 05 |        |      |      |      |    |
| VPN = 06 |        | a[0] | a[1] | a[2] |    |
| VPN = 07 | a[3]   | a[4] | a[5] | a[6] |    |
| VPN = 08 | a[7]   | a[8] | a[9] |      |    |
| VPN = 09 |        |      |      |      |    |
| VPN = 10 |        |      |      |      |    |
| VPN = 11 |        |      |      |      |    |
| VPN = 12 |        |      |      |      |    |
| VPN = 13 |        |      |      |      |    |
| VPN = 14 |        |      |      |      |    |
| VPN = 15 |        |      |      |      |    |

## Ejemplo - Accediendo a un arreglo

A continuación se muestra como una TLB puede mejorar el desempeño en la traducción de direcciones

```
1  int sum = 0;
2  for (i = 0; i < 10; i++) {
3      sum += a[i];
4  }
```

## Suposiciones:

### 4. Páginas:

- $\text{size}(\text{page}) = 16 \text{ B}$
- $\text{num\_paginas} = \text{size}(\text{VA}) / \text{size}(\text{page}) = 256 / 16 = 16$

# Address Translation usando TLB

|          | OFFSET |      |      |      |    |
|----------|--------|------|------|------|----|
|          | 00     | 04   | 08   | 12   | 16 |
| VPN = 00 |        |      |      |      |    |
| VPN = 01 |        |      |      |      |    |
| VPN = 02 |        |      |      |      |    |
| VPN = 03 |        |      |      |      |    |
| VPN = 04 |        |      |      |      |    |
| VPN = 05 |        |      |      |      |    |
| VPN = 06 |        | a[0] | a[1] | a[2] |    |
| VPN = 07 | a[3]   | a[4] | a[5] | a[6] |    |
| VPN = 08 | a[7]   | a[8] | a[9] |      |    |
| VPN = 09 |        |      |      |      |    |
| VPN = 10 |        |      |      |      |    |
| VPN = 11 |        |      |      |      |    |
| VPN = 12 |        |      |      |      |    |
| VPN = 13 |        |      |      |      |    |
| VPN = 14 |        |      |      |      |    |
| VPN = 15 |        |      |      |      |    |

## Ejemplo - Accediendo a un arreglo

### Pregunta:

- ¿Cual es el offset asociado a la dirección de 100?

# Address Translation usando TLB

|          | OFFSET |      |      |      |    |
|----------|--------|------|------|------|----|
|          | 00     | 04   | 08   | 12   | 16 |
| VPN = 00 |        |      |      |      |    |
| VPN = 01 |        |      |      |      |    |
| VPN = 02 |        |      |      |      |    |
| VPN = 03 |        |      |      |      |    |
| VPN = 04 |        |      |      |      |    |
| VPN = 05 |        |      |      |      |    |
| VPN = 06 |        | a[0] | a[1] | a[2] |    |
| VPN = 07 | a[3]   | a[4] | a[5] | a[6] |    |
| VPN = 08 | a[7]   | a[8] | a[9] |      |    |
| VPN = 09 |        |      |      |      |    |
| VPN = 10 |        |      |      |      |    |
| VPN = 11 |        |      |      |      |    |
| VPN = 12 |        |      |      |      |    |
| VPN = 13 |        |      |      |      |    |
| VPN = 14 |        |      |      |      |    |
| VPN = 15 |        |      |      |      |    |

## Ejemplo - Accediendo a un arreglo

### Pregunta:

- ¿Cual es el offset asociado a la dirección de 100?

### Solución:

- $\text{size}(\text{page}) = 16 \rightarrow n(\text{offset}) = \log_2(16) = \log_2(2^4) = 4$
- $n(\text{VA}) = 8$
- $n(\text{VPN}) = n(\text{VA}) - n(\text{offset}) = 8 - 4 = 4$

$$\text{VA} = 100 = \text{01100100} = \text{01100100}$$

- $\text{VPN} = 0110 = 6$
- $\text{offset} = 0100 = 4$

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |

# Address Translation usando TLB

|          | OFFSET |      |      |      |    |
|----------|--------|------|------|------|----|
|          | 00     | 04   | 08   | 12   | 16 |
| VPN = 00 |        |      |      |      |    |
| VPN = 01 |        |      |      |      |    |
| VPN = 02 |        |      |      |      |    |
| VPN = 03 |        |      |      |      |    |
| VPN = 04 |        |      |      |      |    |
| VPN = 05 |        |      |      |      |    |
| VPN = 06 |        | a[0] | a[1] | a[2] |    |
| VPN = 07 | a[3]   | a[4] | a[5] | a[6] |    |
| VPN = 08 | a[7]   | a[8] | a[9] |      |    |
| VPN = 09 |        |      |      |      |    |
| VPN = 10 |        |      |      |      |    |
| VPN = 11 |        |      |      |      |    |
| VPN = 12 |        |      |      |      |    |
| VPN = 13 |        |      |      |      |    |
| VPN = 14 |        |      |      |      |    |
| VPN = 15 |        |      |      |      |    |

## Ejemplo - Accediendo a un arreglo

### Pregunta:

- Asumiendo una TLB de una sola entrada, ¿Cuántos **TLB miss** y **TLB hits** se presentan en este ejemplo?

| TLB |     | PT  |       |
|-----|-----|-----|-------|
| VPN | PFN | VPN | PFN   |
|     |     | ... | ...   |
|     |     | 6   | PFN_x |
|     |     | 7   | PFN_y |
|     |     | 8   | PFN_z |
|     |     | ... | ...   |

| Dato                         | VPN | Resultado (M/H) |
|------------------------------|-----|-----------------|
| a[0]<br>a[1]<br>a[2]         |     |                 |
| a[3]<br>a[4]<br>a[5]<br>a[6] |     |                 |
| a[7]<br>a[8]<br>a[9]         |     |                 |

# Address Translation usando TLB

|          | 00   | 04   | 08   | 12   | 16 |
|----------|------|------|------|------|----|
| VPN = 00 |      |      |      |      |    |
| VPN = 01 |      |      |      |      |    |
| VPN = 02 |      |      |      |      |    |
| VPN = 03 |      |      |      |      |    |
| VPN = 04 |      |      |      |      |    |
| VPN = 05 |      |      |      |      |    |
| VPN = 06 |      | a[0] | a[1] | a[2] |    |
| VPN = 07 | a[3] | a[4] | a[5] | a[6] |    |
| VPN = 08 | a[7] | a[8] | a[9] |      |    |
| VPN = 09 |      |      |      |      |    |
| VPN = 10 |      |      |      |      |    |
| VPN = 11 |      |      |      |      |    |
| VPN = 12 |      |      |      |      |    |
| VPN = 13 |      |      |      |      |    |
| VPN = 14 |      |      |      |      |    |
| VPN = 15 |      |      |      |      |    |

## Ejemplo - Accediendo a un arreglo

### Pregunta:

- Asumiendo una TLB de una sola entrada, ¿Cuántos **TLB miss** y **TLB hits** se presentan en este ejemplo?

| TLB |       |      | PT  |       |
|-----|-------|------|-----|-------|
| VPN | PFN   |      | VPN | PFN   |
| 3   | 3     |      | ... | ...   |
| 6   | PFN_x | M    | 6   | PFN_x |
| 7   | PFN_y | HHM  | 7   | PFN_y |
| 8   | PFN_z | HHHM | 8   | PFN_z |
|     |       | HH   | ... | ...   |

M: Miss (3)

H: Hit (7)

| Dato | VPN | Resultado (M/H) |
|------|-----|-----------------|
| a[0] | 6   | M               |
| a[1] | 6   | H               |
| a[2] | 6   | H               |
| a[3] | 7   | M               |
| a[4] | 7   | H               |
| a[5] | 7   | H               |
| a[6] | 7   | H               |
| a[7] | 8   | M               |
| a[8] | 8   | H               |
| a[9] | 8   | H               |

# Address Translation usando TLB

## Ejemplo - Accediendo a un arreglo

### Pregunta:

- Asumiendo una TLB de una sola entrada, ¿Cuantos **TLB miss** y **TLB hits** se presentan en este ejemplo?

| TLB |       | PT  |       |
|-----|-------|-----|-------|
| VPN | PFN   | VPN | PFN   |
| ?   | ?     | ... | ...   |
| 6   | PFN_x | 6   | PFN_x |
| 7   | PFN_y | 7   | PFN_y |
| 8   | PFN_z | 8   | PFN_z |
|     |       | ... | ...   |

M

HHM

HHHM

HH

**M:** Miss (3)

**H:** Hit (7)

| Acceso a[i] | 0         | 1 | 2 | 3                  | 4 | 5 | 6 | 7 | 8 | 9 |
|-------------|-----------|---|---|--------------------|---|---|---|---|---|---|
| H/M         | M         | H | H | M                  | H | H | H | M | H | H |
| VPN         | 6         |   |   | 7                  |   |   |   | 8 |   |   |
| Hits        | 7 (70%)   |   |   | Estadísticas       |   |   |   |   |   |   |
| Miss        | 3 (30%)   |   |   |                    |   |   |   |   |   |   |
| Total       | 10 (100%) |   |   | TLB Hit rate = 70% |   |   |   |   |   |   |

### Conclusión

Las TLB mejoran el desempeño debido a los principios de localidad espacial y temporal

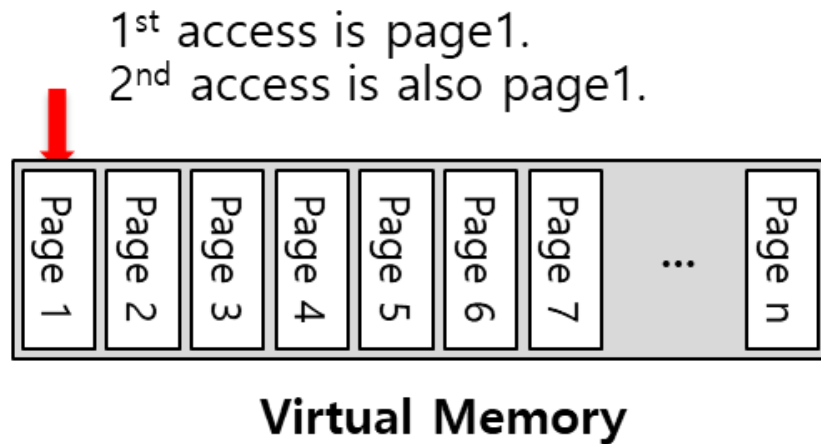
# Address Translation usando TLB

## Principios de localidad

### Localidad Temporal

Una instrucción o dato que ha sido accedido recientemente es probable que vuelva a ser accedido pronto.

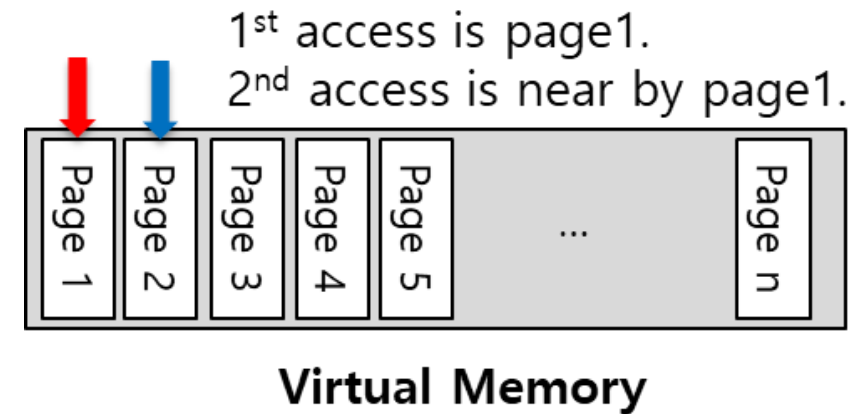
- **Ejemplo:** Ciclo



### Localidad espacial

Si un programa accede a una dirección de memoria x, probablemente pronto accederá a una dirección cercana a x.

- **Ejemplo:** Array



Al ser pequeñas y rápidas las memoria caché aprovechan estos principios.

# Address Translation usando TLB

## Importancia del tamaño de la página

- A mayor tamaño de la página, menor cantidad de TLB misses.

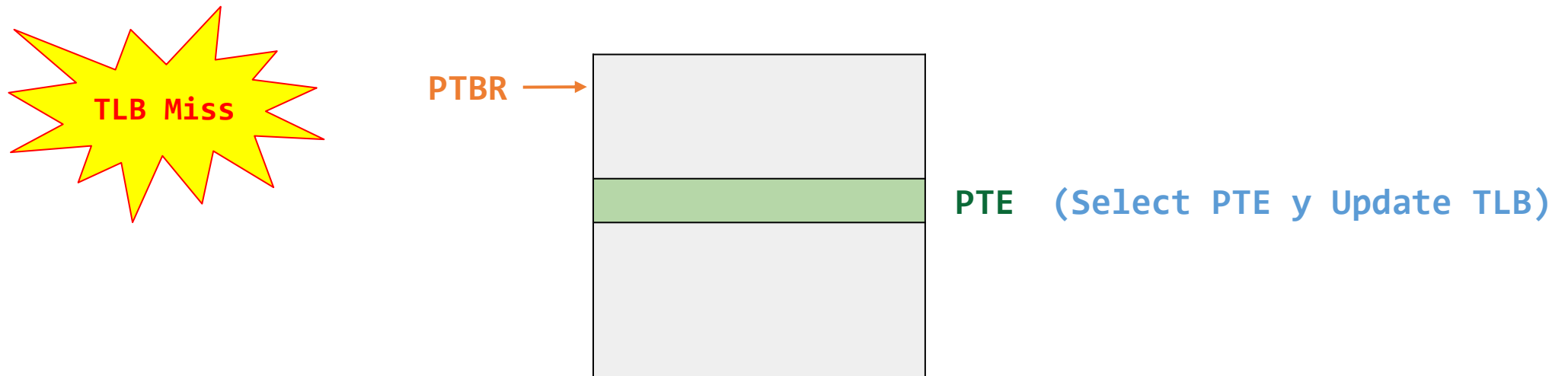
| Page Size | TLB Misses | Hit Rate |
|-----------|------------|----------|
| 16        | 3          | 70 %     |
| 32        | 2          | 80 %     |

- Con un tamaño de página grande (por ejemplo 4KB), la tasa de Hits mejora y por lo tanto, también el desempeño. (Hit rate → 100%).

# ¿Quién maneja las TLB Miss?

## Opción 1 - Hardware-managed TLB (x86, ARM)

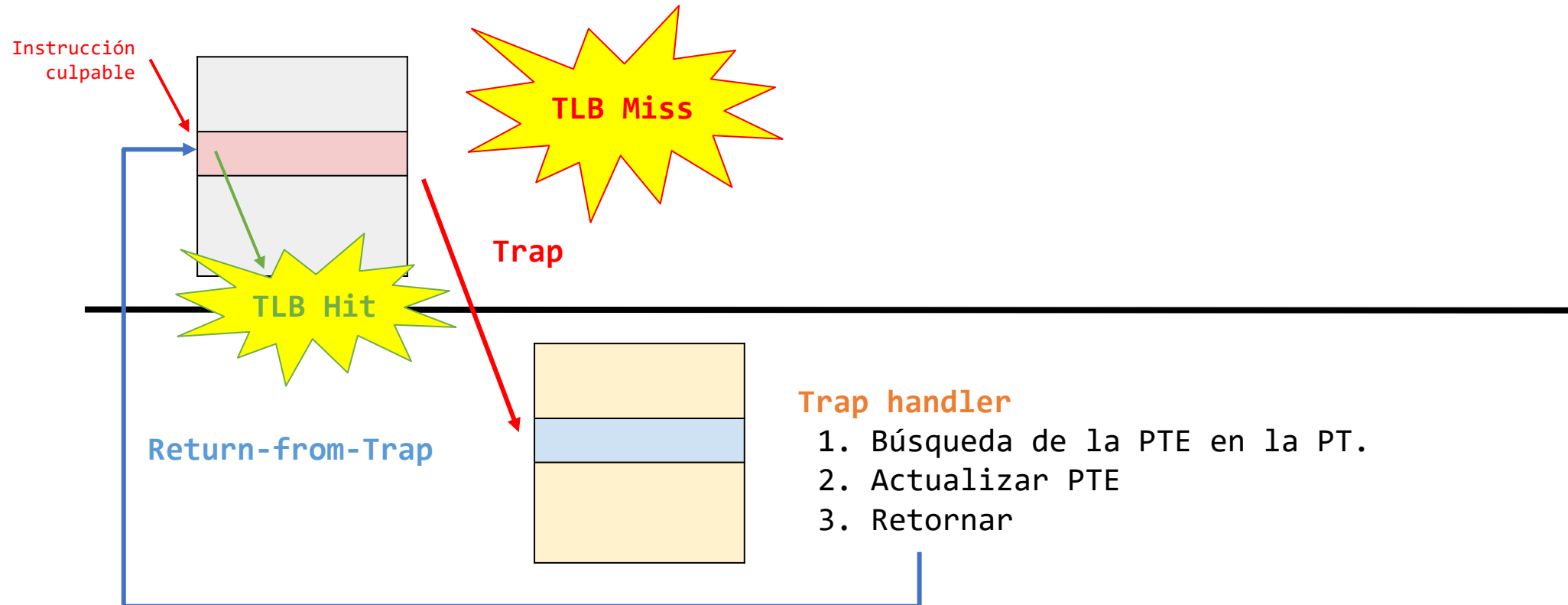
- El hardware conoce donde se encuentran las Page tables (PT) en memoria.
- Cuando sucede un TLB Miss, el Hardware se **mueve** a través de la PT, **encuentra** la PTE correcta y **extrae** la traducción deseada para luego **actualizar** la TLB. Luego la instrucción se vuelve a ejecutar.
- El hardware especifica el formato exacto de la PT.
- Uso de los registros de Hardware (PTBR).



# ¿Quién maneja las TLB Miss?

## Opción 2 - Software-managed TLB (MIPS,...)

- Cuando sucede un TLB miss, el hardware lanza la excepción.
- La excepción es manejada por el Trap handler.
- En este caso la instrucción **return to trap** es un poco **diferente** al caso tradicional ya que en este caso el hardware debe continuar la ejecución en la instrucción causante del trap (y no después).



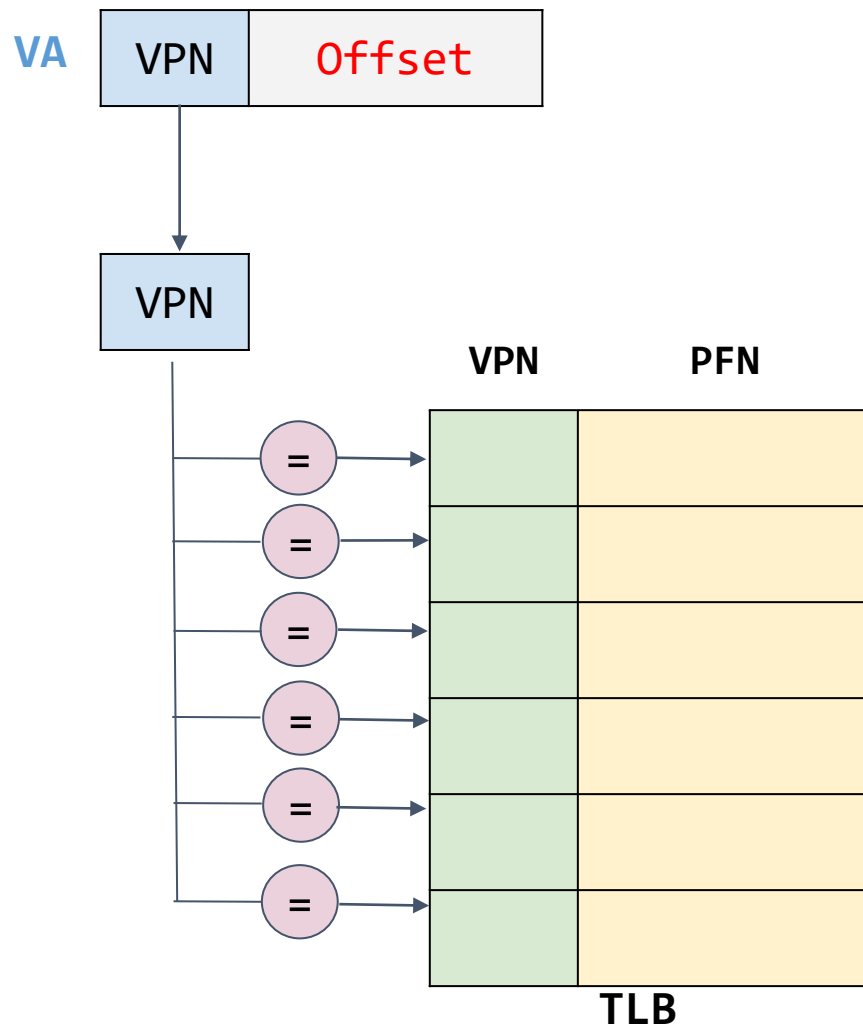
# ¿Quién maneja las TLB Miss?

## Opción 2 - Software-managed TLB (MIPS,...)

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True) // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else // TLB Miss
11     RaiseException(TLB_MISS)
```

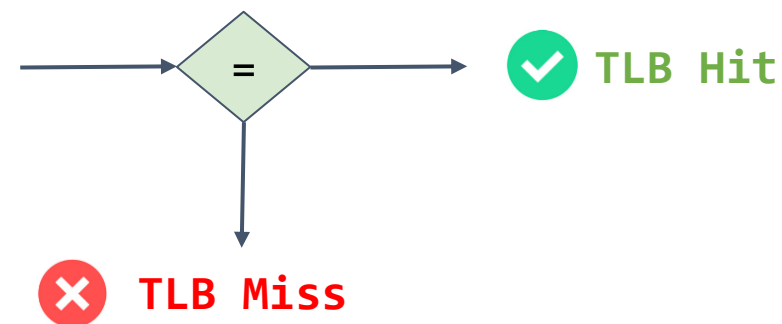
**TLB Control Flow Algorithm (OS Handled)**

# Contenido de una entrada TLB



## Sobre las TLB

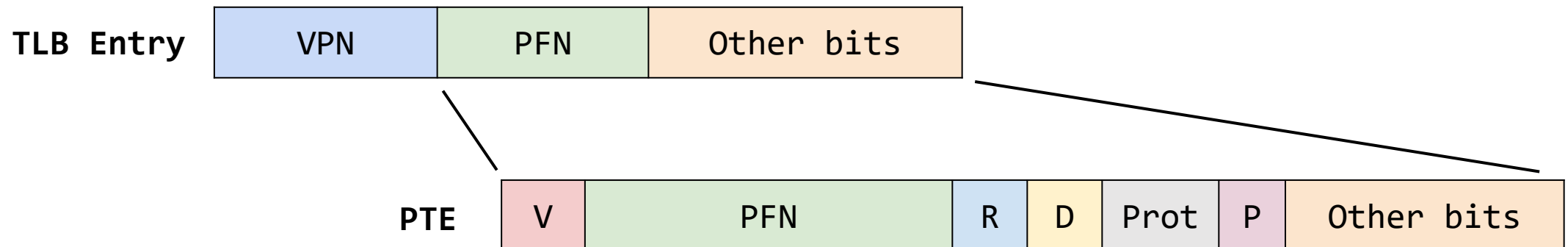
- Memoria cache completamente asociativa.
- Cualquier traducción puede estar en cualquier lugar de la TLB (aleatoria).
- La búsqueda se hace de manera paralela.
- Una TLB típica puede tener 32, 64 o 128 entradas (TLB entry)



# Contenido de una entrada TLB

## Entrada TLB

- **V (Valid)**: bit que dice si un PTE puede o no ser usada. Se evalúa cada vez que una dirección virtual es usada.
- **D (Dirt)**: Indica si la salida ha sido modificada (Una operación write ocurre sobre esta).
- **R (Reference)**: Indica si la página ha sido accedida. Es llevada a 1 (set) cuando la página ha sido leída o escrita. Es útil ya que:
  - Rastrea el acceso a la página.
  - Determina la popularidad de la página.
- **P (Protection)**: Bits que controlan las operaciones que son permitidas (R, W, X, User/Kernel, etc).
- **P (Present)**: Indica si la página se encuentra en memoria física o en el disco.
- **ASID (Address Space Identifier)**: Lo veremos en breve.



# TLB y Context switch

- El TLB es una estructura de hardware (memoria cache) **compartida por todos los procesos**.
- Las traducciones para el proceso en ejecución, **no tienen sentido** para los demás procesos.
- Es necesario que cuando se cambie de un proceso a otro; el hardware, el software o ambos, tengan en cuenta este detalle para asegurarse de que el proceso que está a punto de ejecutarse, no acceda de manera accidental la información de algún proceso ejecutado previamente.

# TLB y Context switch

## Ejemplo:

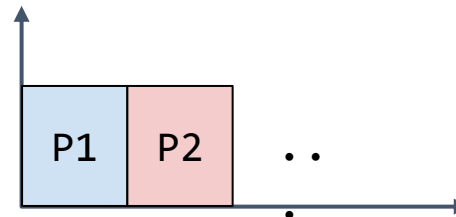
Se tienen dos procesos (P1 y P2) con las siguientes PT y la siguiente TLB.

|    |       |     |
|----|-------|-----|
| P1 | PT P1 |     |
|    | VPN   | PFN |
|    | 10    | 100 |

|    |       |     |
|----|-------|-----|
| P2 | PT P2 |     |
|    | VPN   | PFN |
|    | 10    | 170 |

¿Que pasa cuando se da un cambio de contexto?

**Problema:** Se tiene la misma tabla para todos los procesos; sin embargo, no se puede distinguir qué entrada se asocia a cada proceso.



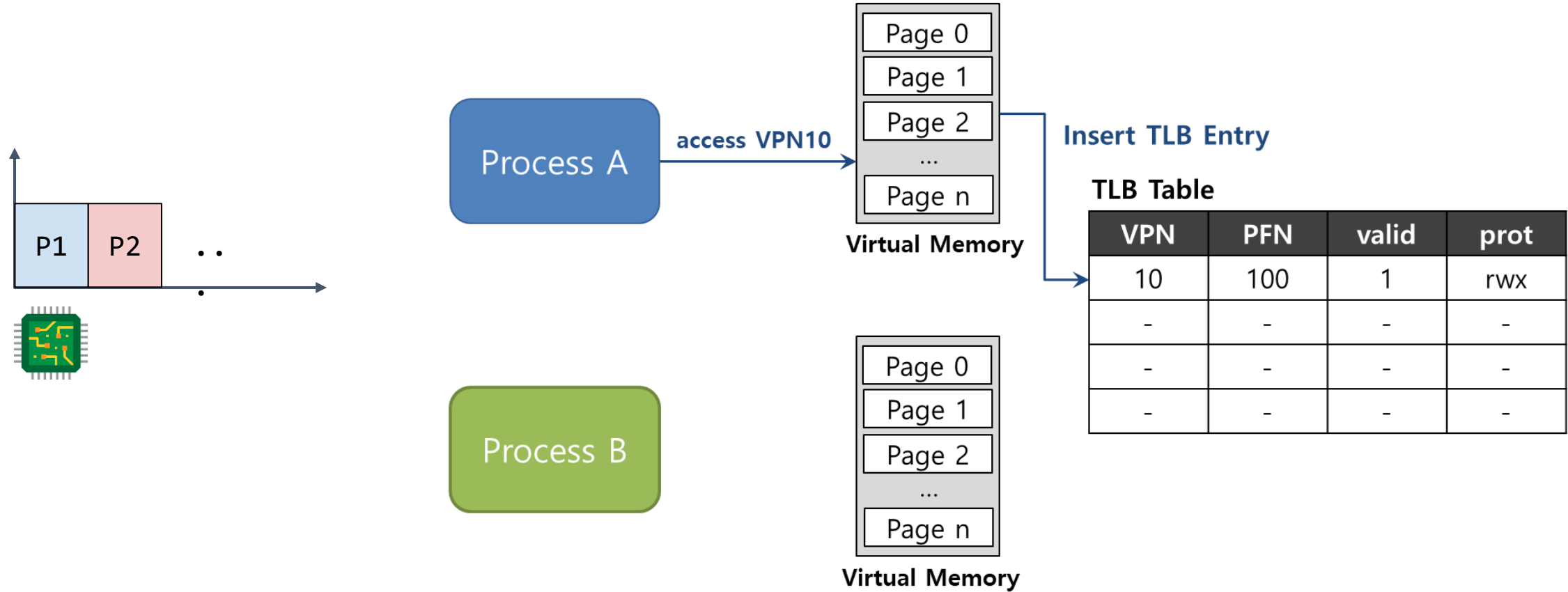
| VPN | PFN | valid | Prot |
|-----|-----|-------|------|
| 10  | 100 | 1     | rwX  |
| --- | --- | 0     | ---  |
| 10  | 170 | 1     | rwX  |
| --- | --- | 0     | ---  |

?

?

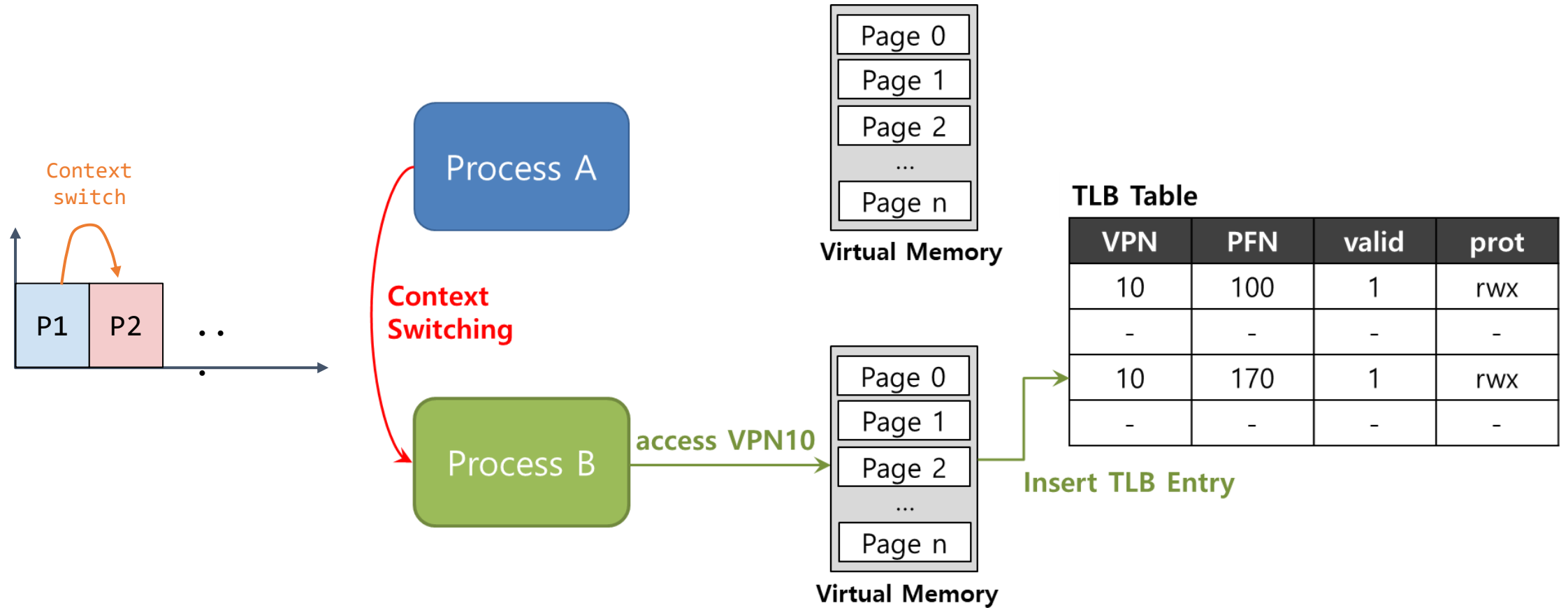
# TLB y Context switch

## Paso a paso



# TLB y Context switch

## Paso a paso



# TLB y Context switch

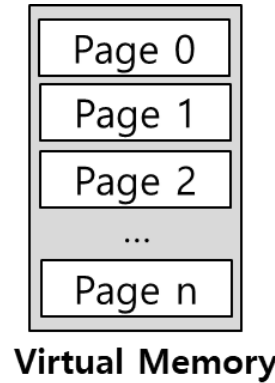
## Paso a paso

### Problema:

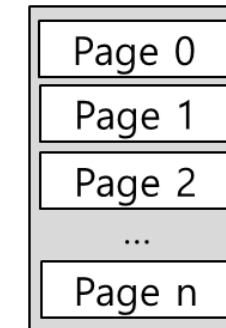
Ambos procesos tienen entradas en la TLB pero no se puede distinguir cuál entrada está asociada a cada proceso.

Process A

Process B



Virtual Memory



Virtual Memory

TLB Table

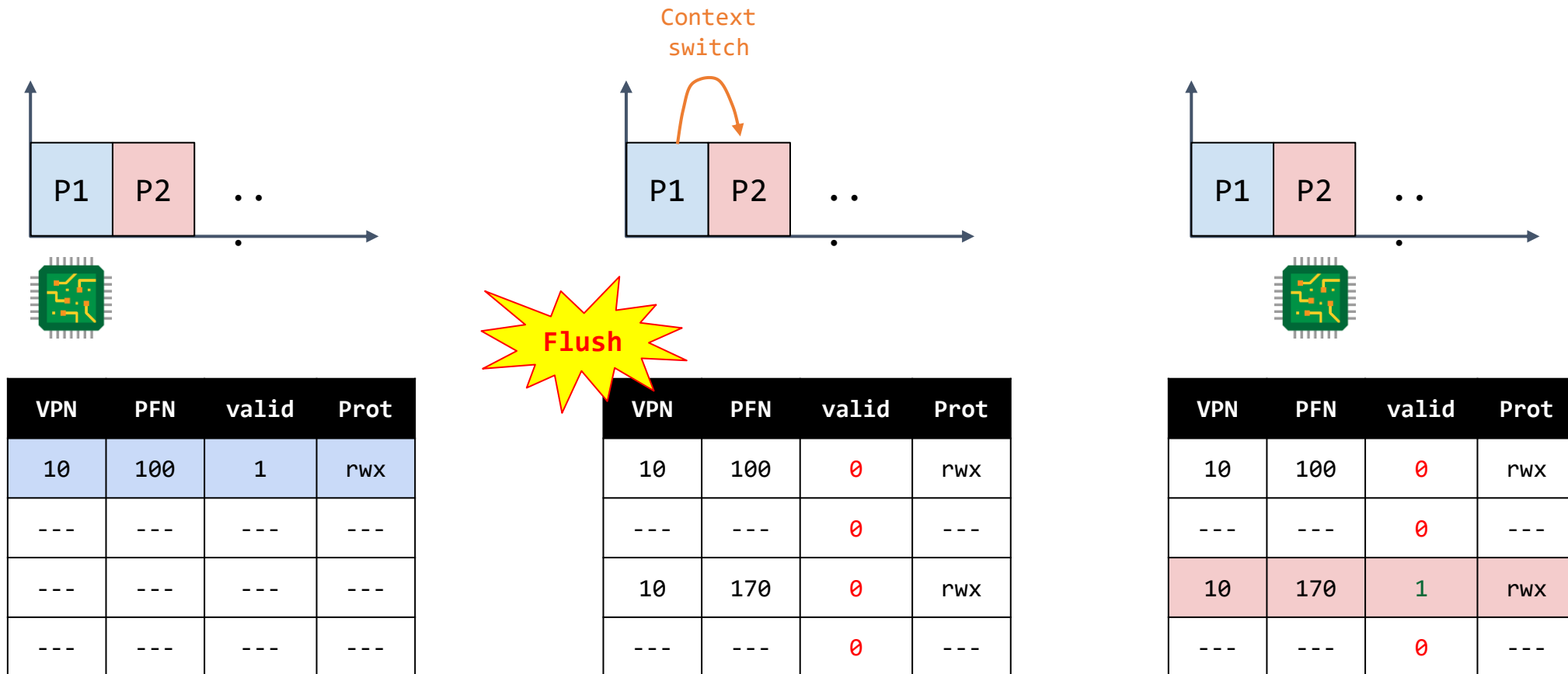
| VPN | PFN | valid | prot |
|-----|-----|-------|------|
| 10  | 100 | 1     | rwX  |
| -   | -   | -     | -    |
| 10  | 170 | 1     | rwX  |
| -   | -   | -     | -    |

# TLB y Context switch

## Solucionando el problema

### Solución 1 - Vaciar (flush) la TLB en los cambios de contexto

- **Flush:** Establecer todos los bits de validez (**V**) a **cero** (0).



# TLB y Context switch

## Solucionando el problema

### Solución 1 - Vaciado (vaciado) de la TLB - Consecuencias del vaciado

1. Un proceso nunca encontrará accidentalmente traducciones incorrectas en la TLB.
2. El **flushing** es costoso pues se pierden todas las traducciones recientemente cargadas a la TLB:

context switch [up] → TLB miss [up] → Costo [up]

# TLB y Context switch

## Solucionando el problema

### Solución 2 - Proveer un campo Address Space Identifier (ASID) en el TLB

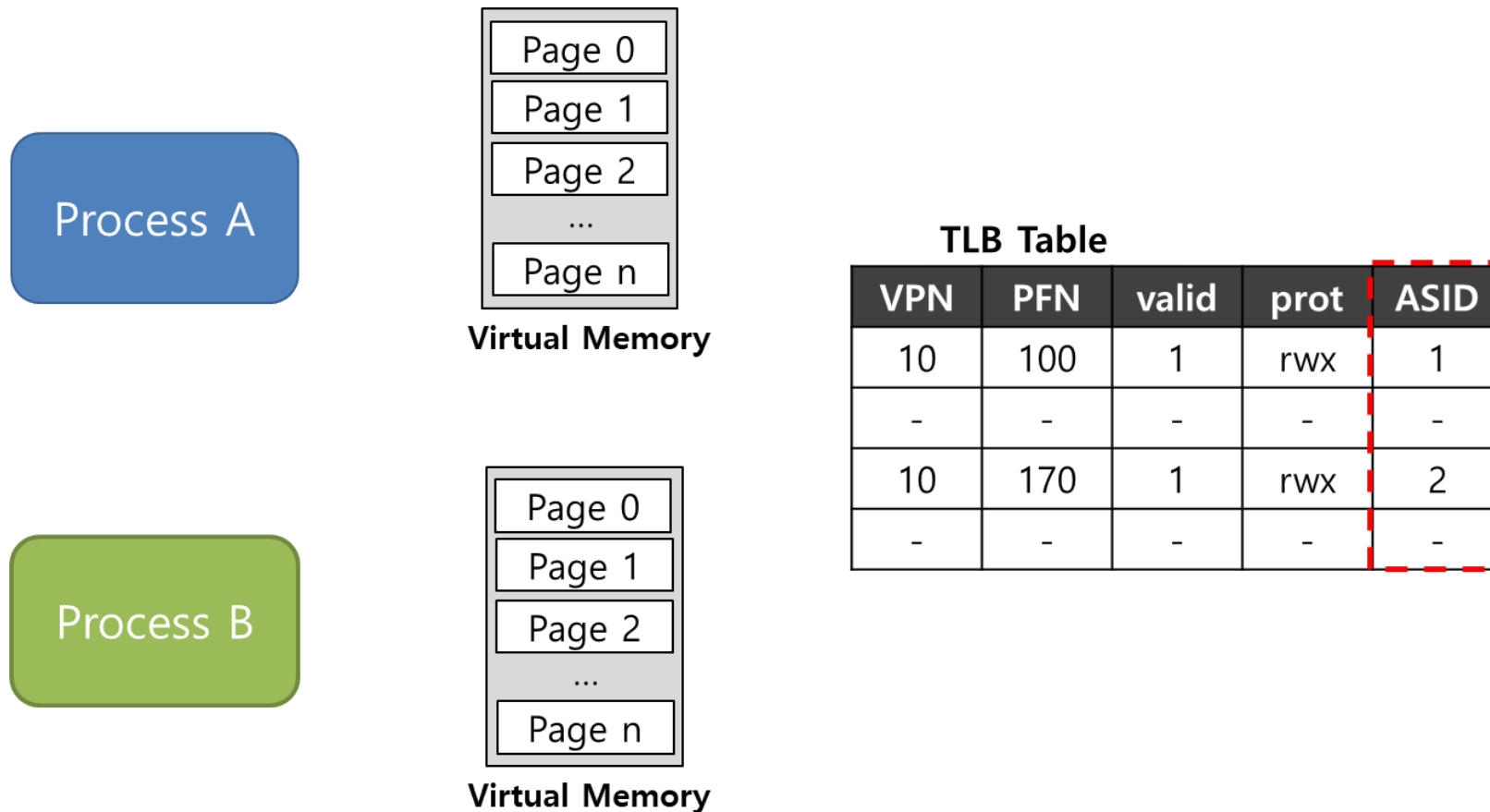
- **ASID (Address Space Identifier):** Campo de 8 bits en la TLB.
- Permite diferenciar un proceso de otro en la TLB.
- Gracias a este campo, varios procesos pueden compartir la TLB (manteniendo sus instrucciones) sin ninguna confusión.
- Soluciona el problema de overhead debido a la técnica de vaciado (flushing) de la TLB.

| VPN | PFN | valid | Prot | ASID |
|-----|-----|-------|------|------|
| --- | --- | ---   | ---  | ---  |
| --- | --- | ---   | ---  | ---  |
| --- | --- | ---   | ---  | ---  |
| --- | --- | ---   | ---  | ---  |

# TLB y Context switch

## Solucionando el problema

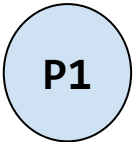
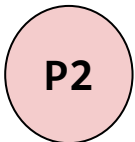
Proveer un campo Address Space Identifier (ASID) en el TLB - Ejemplo



# TLB y Context switch

## Solucionando el problema

### Proveer un campo Address Space Identifier (ASID) en el TLB - Ejemplo

|                                                                                   |     |                                                                                   |     |
|-----------------------------------------------------------------------------------|-----|-----------------------------------------------------------------------------------|-----|
|  |     |  |     |
| PT P1                                                                             |     | PT P2                                                                             |     |
| VPN                                                                               | PFN | VPN                                                                               | PFN |
| 10                                                                                | 101 | 50                                                                                | 101 |

| VPN | PFN | valid | Prot | ASID |
|-----|-----|-------|------|------|
| 10  | 101 | 1     | rwX  | 1    |
| --- | --- | 0     | ---  | ---  |
| 50  | 101 | 1     | rwX  | 2    |
| --- | --- | 0     | ---  | ---  |

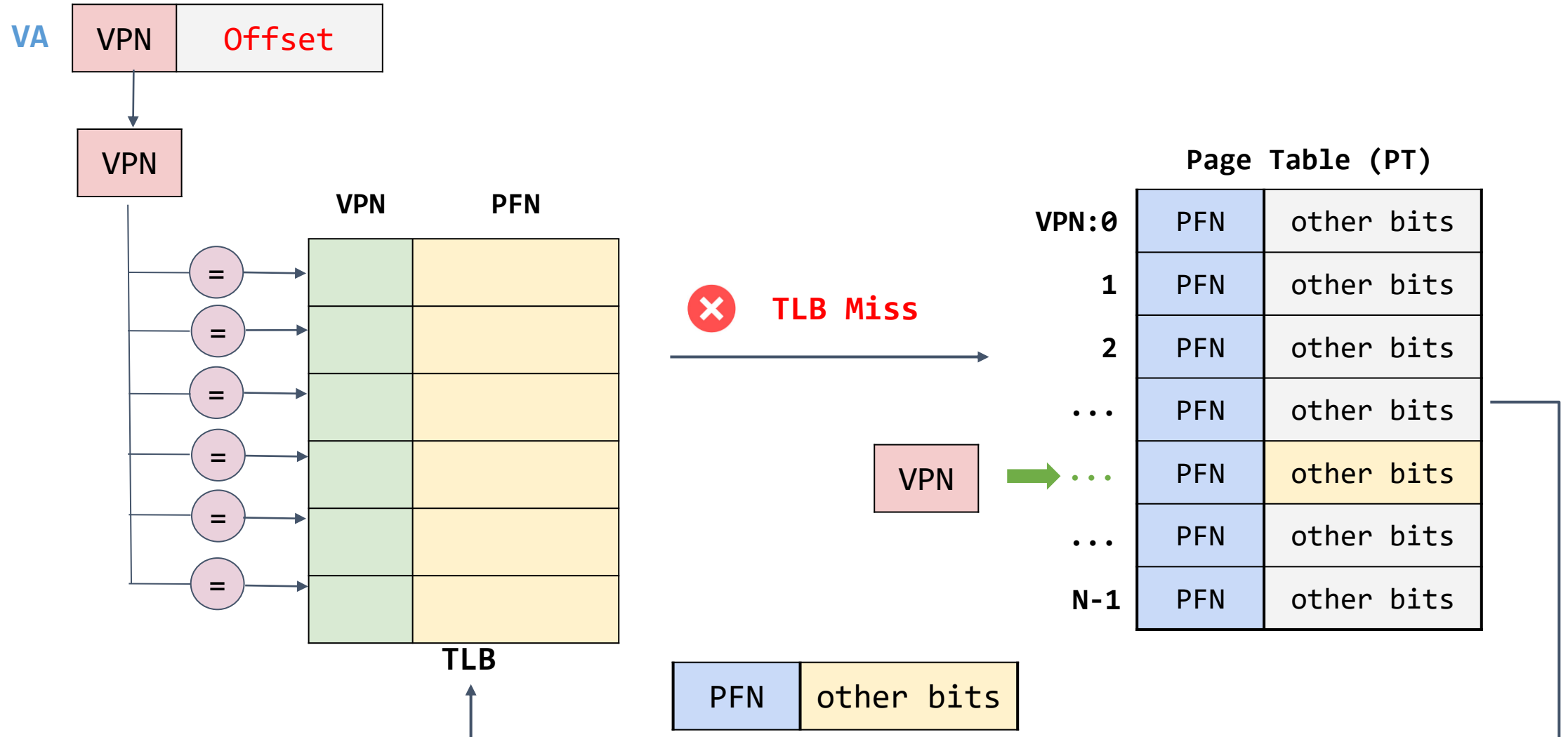
- Es común que procesos distintos compartan segmentos de código (binaries, shared libraries).

¿Qué pasa cuando dos procesos apuntan a un mismo frame (PFN)?

Compartir páginas es **útil** porque reduce el número de frames en uso.

# Políticas de reemplazo

Una **política de reemplazo** en este caso, consiste en el procedimiento llevado a cabo para reemplazar una entrada antigua en la TLB (cache) por una nueva.



# Políticas de reemplazo

## Objetivo de la política de reemplazo

Reducir la tasa de miss aumentando por consiguiente la tasa de hits.



## ¿Cómo hacer el reemplazo?

Hay diferentes políticas, algunas son:

1. Least Recently Used (LRU).
2. Random (aleatorio).

# Políticas de reemplazo

## LRU (Least Recently Used)

Reemplaza la entrada de la TLB que no haya sido usada recientemente (la que lleva más tiempo sin ser accedida).

### Ejemplo:

Asuma que se tiene una TLB de tres entradas, y que el acceso a memoria (principal) se está realizando de acuerdo al orden presentado en la siguiente traza:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2

Reference row → 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2

Page Frame →



# Políticas de reemplazo

## LRU (Least Recently Used)

### Ejemplo:

Asuma que se tiene una TLB de tres entradas, y que el acceso a memoria (principal) se está realizando de acuerdo al orden presentado en la siguiente traza:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2

|               |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Reference row | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 | 1 | 2 |
|               | → |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| Page Frame →  | 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 | 0 | 1 | 1 |
|               |   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 3 | 3 | 3 | 3 | 3 |
|               |   |   | 1 | 1 | 1 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
|               | M | M | M | M | H | M | H | M | M | M | M | H | H | M | H |

# Políticas de reemplazo

## Random

Realiza un reemplazo seleccionando una entrada al azar de la TLB.

## Ejemplo:

Asuma que se tiene una TLB de tres entradas, y que el acceso a memoria (principal) se está realizando de acuerdo al orden presentado en la siguiente traza:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2

Reference row → 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2

Page Frame →



# Políticas de reemplazo

## Random

### Ejemplo:

Asuma que se tiene una TLB de tres entradas, y que el acceso a memoria (principal) se está realizando de acuerdo al orden presentado en la siguiente traza:

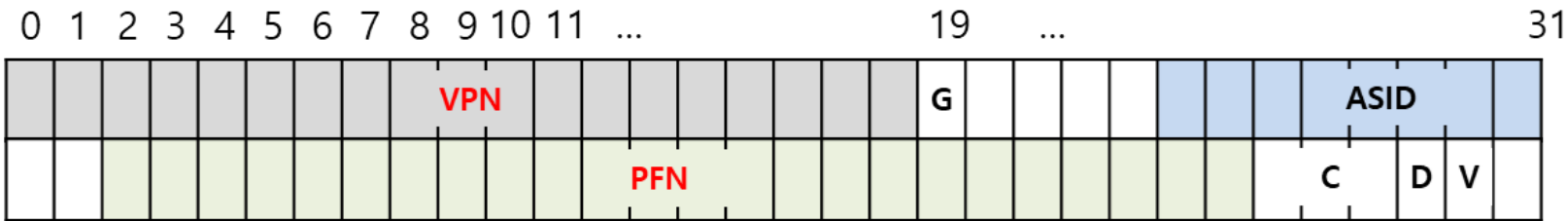
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2

|               |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Reference row | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 | 1 | 2 |
| →             |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| Page Frame →  | 7 | 7 | 7 | 7 | 7 | 3 | 3 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |
|               |   |   | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 3 | 3 | 3 | 2 | 2 | 2 |
|               |   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
|               | M | M | M | M | H | M | H | M | H | M | H | H | H | M | H |
|               | 0 | 2 | 1 | 1 |   | 0 |   | 0 |   | 1 |   |   |   | 2 |   |

# Entrada TLB real - caso MIPS

## Forma de la entrada TLB

- TLB de un MIPS R4000.
- El sistema operativo maneja la TLB por software.



All 64 bits of this TLB entry (example of MIPS R4000)

| Flag              | Content                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------|
| 19-bit VPN        | The rest reserved for the kernel.                                                       |
| 24-bit PFN        | Systems can support with up to 64GB of main memory( $2^{24} \times 4\text{KB}$ pages ). |
| Global bit (G)    | Used for pages that are globally-shared among processes.                                |
| ASID              | OS can use to distinguish between address spaces.                                       |
| Coherence bit (C) | Determine how a page is cached by the hardware.                                         |
| Dirty bit (D)     | Marking when the page has been written.                                                 |
| Valid bit (V)     | Tells the hardware if there is a valid translation present in the entry.                |

# Entrada TLB real - caso MIPS

## Sobre el MIPS R4000

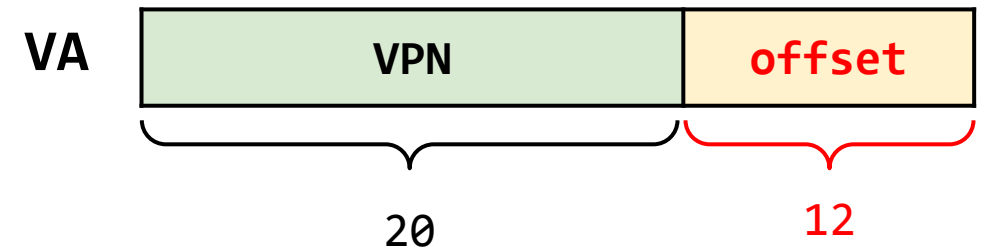
- Soporta un espacio de direccionamiento de 32 bits ( $n(AS) = 32$ ).
- El tamaño de las páginas es de 4 KB ( $size(page) = 4KB$ ).

### Tamaño de la memoria virtual:

- $size(AS) = 2^{n(AS)} = 2^{32}$   
 $= (2^2)(2^{30}) = 4 \text{ GB}$

### Número de páginas:

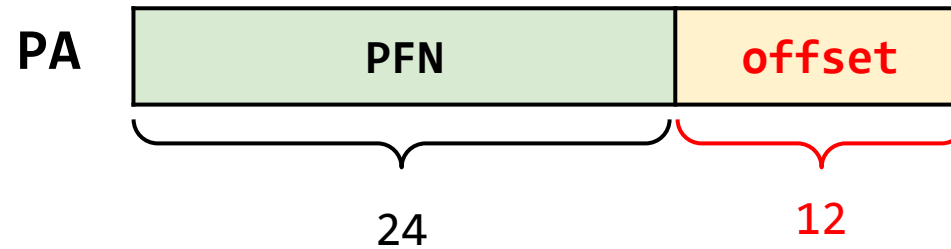
- $num\_pages = size(AS)/size(page)$   
 $= 2^{32}/(4*(2^{10})) = 2^{20} \text{ pages}$
- Para el caso del **MIPS R4000** el **VPN** es de 19 bits:
  - Espacio kernel: 2 GB.
  - Espacio de direcciones (usuario): 2 GB.



# Entrada TLB real - caso MIPS

## Sobre el MIPS R4000

- La PFN tiene 24 bits para el MIPS R4000, de modo que la forma de una dirección física será:



- De modo que el tamaño de la memoria física es:

$$\text{size(PM)} = 2^{n(\text{PM})} = 2^{36} = (2^6)(2^{30}) = 64 \text{ GB}$$

$$\text{num\_frames} = \text{size(PM)} / \text{size(frame)} = (2^{36}) / (4 * (2^{10})) = (2^{24}) \text{ frames}$$

# Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulos [1](#) y [2](#))
- Videos de youtube: Clase 1 - Introducción a los sistemas operativos ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- <http://pages.cs.wisc.edu/~bart/537/lecturenotes/titlepage.html>
- <https://www.scs.stanford.edu/22wi-cs212/>
- <https://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html>
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/5-Memory-Management.md>
- <https://applied-programming.github.io/Operating-Systems-Notes/5-Memory-Management/>